



UNIVERSITY
of
GREENWICH



AI-Driven Prediction Model for Car Pricing and Classification Based on Features and Market

by

Mostafa Hisham 220367

My Teammate

Hisham Ahmed 221035

A dissertation submitted in partial fulfillment of the
requirements for the degree of
Bachelor of computer science (SE program)

in the

Faculty of Computer Science

of the

October University for Modern Sciences and Arts (MSA), EGYPT

Graduation Projects advisor:
Dr.Mohamed Labib

(July 2025)

Abstract

In Egypt, the used car market is frequently impacted by inflation and false pricing by sellers, who could list cars at either too high or too low rates to attract buyers' attention. This impacts the standard of a fair price for buyers. Our artificial intelligence system's **image identification** and **pricing prediction** assist us in solving this challenge. Using several techniques, we gathered our dataset from 23,421 **Dubizzle** vehicle ads. **XGBoost** has the highest **accuracy**—**97%** rate. Using **DenseNet 201**—trained on 16,185 photos from the **Stanford Cars Dataset**—we have also developed an **image recognition** ML model to detect the car models from images with an accuracy of **94%**. Those without familiarity with car specifics will especially find this feature very useful. Without having to enter all the car's details, which can be challenging and result in errors, this feature helps one receive realistic price ranges. We tested the **pricing prediction** algorithms using the **CarDekho dataset** from **Kaggle** before gathering our data. This allowed us to check the reliability of our system and enhance its accuracy before its implementation by using our data. We have also apply car damage detection model using carDD dataset and with the employing yolo v8 seg model and dcn+ as resnet101 as a backbone model this can help buyers know more about the condition of the car. Additionally, for individuals who know little about vehicles, our system is simple and reliable since we keep it updated to manage changes in the market. This tool enables buyers in an unpredictable market to find an acceptable, basic car price.

Acknowledgments

I would want to thank **Dr. Mohamed Labib**, my supervisor, for his kind advice and support during this project. His advice and assistance really simplified my project work. I also appreciate my doctor's helpful advice and support over this project.

Contents

Acknowledgments	iv
List of Tables	5
List of Figures	7
1 Introduction	8
1.1 Introduction	8
1.1.1 Aims of the Project	8
1.1.2 Intended Audience and Scope	8
1.1.3 Approach	9
1.1.4 Assumptions	9
1.1.5 Outcomes	9
1.2 Problem Definition	9
1.3 Motivation	10
1.4 Project Description	11
1.5 Scope	11
2 Background	12
2.1 Literature Review	13
2.2 Summary	18
2.3 Comparison with Proposed Project	19
2.4 Screen Shots from Previous Systems (if needed)	20
2.5 Project Management and Deliverable	20
2.5.1 Tasks and Time Plan	20
2.5.2 Budget and Resource Costs	21
3 Specification - (SRS)	22
3.1 Introduction	22
3.1.1 Purpose of this Document	22
3.1.2 Scope of this Document	22
3.1.3 Overview	22
3.1.4 Business Context	23
3.2 General Description	23

3.2.1	Product Functions	23
3.2.2	Similar System Information	23
3.2.3	User Characteristics	24
3.2.4	User Problem Statement	24
3.2.5	User Objectives	24
3.2.6	General Constraints	24
3.3	Functional Requirements	25
3.4	Interface Requirements	32
3.4.1	User Interfaces	32
3.4.2	Hardware Interfaces	44
3.4.3	Communications Interfaces	44
3.4.4	Software Interfaces	44
3.5	Performance Requirements	45
3.6	Design Constraints	45
3.6.1	Standards Compliance	45
3.6.2	Hardware Limitations	45
3.6.3	Other Constraints	45
3.7	Other Non-Functional Attributes	45
3.7.1	Security	45
3.7.2	Binary Compatibility	46
3.7.3	Reliability	46
3.7.4	Maintainability	46
3.7.5	Portability	47
3.7.6	Extensibility	47
3.7.7	Re-usability	47
3.7.8	Application Compatibility	48
3.7.9	Resource Utilization	48
3.7.10	Serviceability	48
3.7.11	Performance	48
3.7.12	Scalability	49
3.7.13	Usability	49
3.7.14	Availability	49
3.7.15	Data Integrity	50
3.7.16	Error Handling	50
3.7.17	Data Encryption	50
3.7.18	Regulatory Compliance	50
3.7.19	Continuous Monitoring	51
3.7.20	Continuous Improvement	51
3.8	Preliminary Object-Oriented Domain Analysis	51
3.8.1	Inheritance Relationships	52
3.8.2	Class Descriptions	53
3.9	Operational Scenarios	63
3.9.1	Scenario 1: User Registers an Account	63
3.9.2	Scenario 2: User Logs In to the System	64

3.9.3	Scenario 3: User Browses, Filters, and Searches Listings	64
3.9.4	Scenario 4: User Requests a Car Price Prediction	65
3.9.5	Scenario 5: User Detects Car Damage via Image	65
3.9.6	Scenario 6: User Identifies Car Make/Model via Image	66
3.9.7	Scenario 7: User Manages a Car Listing (Create, View, Edit, Delete)	66
3.9.8	Scenario 8: User Manages Profile	67
3.9.9	Scenario 9: User Exchanges Messages	67
3.9.10	Scenario 10: Admin Views Dashboard and Manages Car Listings . .	68
3.9.11	Scenario 11: Admin Scrapes External Data	68
3.9.12	Scenario 12: Admin Views Data Visualization	69
3.9.13	Use Case Diagram	70
3.9.14	Use Case Diagram	71
3.9.15	Use Case Diagram	72
3.10	Preliminary Schedule Adjusted	72
3.11	Preliminary Budget Adjusted	73
3.12	Appendices	73
3.12.1	Definitions, Acronyms, Abbreviations	73
4	Design	74
4.1	Design Guidelines	74
4.2	Introduction	75
4.2.1	Purpose	75
4.2.2	Scope	75
4.2.3	Overview	75
4.2.4	Reference Material	75
4.2.5	Definitions and Acronyms	76
4.3	System Overview	76
4.4	System Architecture	77
4.4.1	Architectural Design	77
4.4.2	Decomposition Description	77
4.4.3	Design Rationale	79
4.5	Data Design	80
4.5.1	Data Description	80
4.5.2	Data Dictionary	81
4.6	Component Design	81
4.6.1	User Registration and Authentication	81
4.6.2	Car Recognition via Image	82
4.6.3	Price Prediction	82
4.6.4	Car Listing Management	82
4.6.5	Messaging System	83
4.6.6	Admin Dashboard Tools	83
4.6.7	Data Scraping and Scheduling	83
4.7	Human Interface Design	84
4.7.1	Overview of User Interface	84
4.7.2	Screen Images	84

4.7.3	Screen Objects and Actions	90
4.8	Requirements Matrix	91
5	Implementation	93
5.1	General Guidelines	93
5.2	Backend Endpoints	93
5.2.1	1. Price Prediction	93
5.2.2	2. Car Recognition	94
5.2.3	3. Car Detection Check	95
5.2.4	4. Damage Report Retrieval	95
5.2.5	5. Car Details API	96
5.3	Machine Learning Training	97
5.4	Common Challenges and Solutions	97
5.5	Deliverables	97
5.6	Summary	98
6	Results and Evaluation	99
6.1	General Rules	99
6.2	Test Cases Overview	99
6.2.1	Sample Test Case Details	100
6.3	Detailed Test Case Specification	100
6.4	Model Performance Metrics	100
6.5	Locust Performance Testing Results	101
6.6	Critical Evaluation	101
6.6.1	Strengths	101
6.6.2	Weaknesses	101
6.6.3	Methodology Reflections	101
6.7	Next Steps for Confidence	102
7	Conclusions and Future Work	103
7.1	Conclusions	103
7.2	Future Work	104
A	Collected Materials	106
	References or Bibliography	107

List of Tables

3.1	Functional Requirement FR-001: User Registration	25
3.2	Functional Requirement FR-002: User Login	25
3.3	Functional Requirement FR-003: User Logout	25
3.4	Functional Requirement FR-004: User Profile Management	25
3.5	Functional Requirement FR-005: Password Recovery	26
3.6	Functional Requirement FR-006: Car Listing Creation	26
3.7	Functional Requirement FR-007: Edit Car Listing	26
3.8	Functional Requirement FR-008: Delete Car Listing	26
3.9	Functional Requirement FR-009: View Car Listings	27
3.10	Functional Requirement FR-010: Browse/Search/Filter Listings	27
3.11	Functional Requirement FR-011: View Listing Details	27
3.12	Functional Requirement FR-012: Car Price Prediction	27
3.13	Functional Requirement FR-013: Car Damage Detection	27
3.14	Functional Requirement FR-014: Car Recognizer	28
3.15	Functional Requirement FR-015: Car Image Validation	28
3.16	Functional Requirement FR-016: Send Message	28
3.17	Functional Requirement FR-017: Retrieve Conversation History	28
3.18	Functional Requirement FR-018: Mark Message as Read	29
3.19	Functional Requirement FR-019: View System Statistics (Admin Dashboard)	29
3.20	Functional Requirement FR-020: Manage Users (Admin Dashboard)	29
3.21	Functional Requirement FR-021: Manage Car Listings (Admin Dashboard)	29
3.22	Functional Requirement FR-022: Data Visualization (Admin Dashboard) .	30
3.23	Functional Requirement FR-023: Highest Car Make Analysis (Admin Data Analysis)	30
3.24	Functional Requirement FR-024: Highest Car Model Analysis (Admin Data Analysis)	30
3.25	Functional Requirement FR-025: Fuel Type Distribution (Admin Dashboard)	30
3.26	Functional Requirement FR-026: Market Trend Analysis (Admin Dashboard)	31
3.27	Functional Requirement FR-027: Data Scraping and Scheduling (Admin) .	31
3.28	Functional Requirement FR-028: Error Handling and Logging	31
3.29	Functional Requirement FR-029: Data Validation for Car Details	31
3.30	Functional Requirement FR-030: Responsive UI/UX Design	31
3.31	Functional Requirement FR-031: Account Deactivation by User	32

3.32	Functional Requirement FR-032: Car Listing Image Management	32
3.33	Non-Functional Requirement NFR-001: Security	45
3.34	Non-Functional Requirement NFR-002: Binary Compatibility	46
3.35	Non-Functional Requirement NFR-003: Reliability	46
3.36	Non-Functional Requirement NFR-004: Maintainability	46
3.37	Non-Functional Requirement NFR-005: Portability	47
3.38	Non-Functional Requirement NFR-006: Extensibility	47
3.39	Non-Functional Requirement NFR-007: Re-usability	47
3.40	Non-Functional Requirement NFR-008: Application Compatibility	48
3.41	Non-Functional Requirement NFR-009: Resource Utilization	48
3.42	Non-Functional Requirement NFR-010: Serviceability	48
3.43	Non-Functional Requirement NFR-011: Performance	48
3.44	Non-Functional Requirement NFR-012: Scalability	49
3.45	Non-Functional Requirement NFR-013: Usability	49
3.46	Non-Functional Requirement NFR-014: Availability	49
3.47	Non-Functional Requirement NFR-015: Data Integrity	50
3.48	Non-Functional Requirement NFR-016: Error Handling	50
3.49	Non-Functional Requirement NFR-017: Data Encryption	50
3.50	Non-Functional Requirement NFR-018: Regulatory Compliance	50
3.51	Non-Functional Requirement NFR-019: Continuous Monitoring	51
3.52	Non-Functional Requirement NFR-020: Continuous Improvement	51
4.1	Definitions and Acronyms	76
4.2	Data Dictionary of Core System Entities	81
4.3	Requirements Traceability Matrix	92

List of Figures

2.1	Example Screenshot from a Previous System	20
2.2	Gantt Chart for Project Plan	20
3.1	Home Page	34
3.2	Registration Page	34
3.3	Login In	35
3.4	API Gateway	35
3.5	Class Diagram	52
3.6	Inheritance Relations	53
3.7	Use Case Diagram for System Operations	70
3.8	Sequence Diagram User	71
3.9	Sequence Diagram Admin	72
3.10	Gantt Chart for Project Plan	73
4.1	Architectural Design	77
4.2	Web-Scraping System Architecture (Top-Level DFD)	78
4.3	Price-Prediction Model Architecture	78
4.4	Image-Detection Model Architecture	79
4.5	Home Page	84
4.6	Registration Page	85
4.7	Login In	85
4.8	SignUp	86
4.9	View Listing Details	86
4.10	Image recognition	87
4.11	Car damage	87
4.12	Price Prediction	88
4.13	Messages Interface	88
4.14	User ListCar Page	89
4.15	User Listing	89
4.16	User Profile	90
4.17	Admin Dashboard	90

Chapter 1

Introduction

1.1 Introduction

In Egypt, determining a reasonable price for used vehicles is challenging since sellers sometimes set prices either too high or too cheap only to get attention; pricing are flexible. This makes it challenging for buyers as well as sellers to decide on acceptable pricing. Our work designs a system able to accurately estimate car cost and identify car models from images using advanced methods.

1.1.1 Aims of the Project

The main goal of this project is to develop an artificial intelligence system able to accurately identify different types of vehicles, car damages and decide their price and give a damage report. The system aims to reduce complexity and enhance the fairness of the buying and selling procedures in Egypt by offering acceptable car pricing.

1.1.2 Intended Audience and Scope

This system is designed for Egyptians who might not know much about cars but want to buy or sell them. It benefits people like them by offering them with an acceptable estimation of the range of prices for vehicles they want to sell or buy. Those who enjoy a car and wish to know its details and price without having thorough understanding about cars will find this extremely useful. The project aims to provide a user-friendly method transforming challenging cars data into understandable facts for everyone.

1.1.3 Approach

Our method uses different technologies and steps:

- **Data Collection:** Using systems to automatically gather this data, we collect it from more than 23,421 cars advertised on websites like Dubizzle.
- **Image Recognition:** Using a model called DenseNet 201, trained on many car images, with an accuracy of 94% our system can identify car models from images for people who know little about car features.
- **Price Prediction:** We analyze car data using XGBoost and other machine learning algorithms that can predict cars prices with 97% accuracy.
- **Damage detection:** We used cardd dataset to give car damage report using yolo v8 seg model and dcn+ resnet101 as a backbone.
- **Deployment:** React is used for the frontend, and FastAPI is used for the backend. This ensures the application is responsive and simple for consumers. Data is stored in MongoDB; Material-UI handles design; Recharts visually show data.

1.1.4 Assumptions

- The data gathered online fairly shows the current car market situation.
- The market for cars is continually changing; so, our system has to be regularly changed to ensure ongoing accuracy.

1.1.5 Outcomes

- By 30 percent as compared to past approaches, our estimate of car pricing shows a significant improvement in accuracy, hence reducing errors.
- More trust in car sales based on accurate, current vehicle estimations.
- For non-car experts, this simpler experience lets customers make wise decisions based on reliable, easily accessible data.

1.2 Problem Definition

- The inconsistent pricing and unreadable used car market that makes it challenging for consumers to determine the fair market worth of cars.

- The difficulty non-experts have in accurately recognizing cars and features, which is essential for wise car purchase decisions.

1.3 Motivation

Due to used car market challenges with accurate car evaluations and model information, the idea came. Seller prices and inflation can mislead car buyers. The used car market is complex; therefore, model, year, and CC affect pricing. Even though some existing systems offer pricing estimates, they normally need customers to enter car details that they may not know. We solve this gap by using image recognition technology to allow consumers to upload a car photo for quick and complete details, including a proper market value. Ken Research's comprehensive research shows consumer desire for clear, trustworthy, and easy-to-use car evaluation tools. This demand highlights the importance and potential market influence of our project, which encourages us to create a solution that improves user experience and supports academic and realistic machine learning and computer vision improvements.

Our technical targets are to address:

1. Creating scalable, accurate machine learning models for price prediction that fit inflation and market changes will help to guarantee that consumers get fair market prices right now.
2. Using advanced image recognition technologies to enable consumers to recognize car types and brands from images will help individuals unfamiliar with cars details.
3. Improving data processing techniques to effectively manage big and different datasets ensures fast and consistent outcomes.
4. Making sure strong data security systems are in place protects user data.
5. developing clear user interfaces for people with different degrees of technological knowledge.
6. building a scalable system architecture able to expand with increasing user demand and enable real-time data updates.
7. Including several data sources helps to keep the system relevant and accurate.
8. Creating cross-platform compatibility ensures functionality on many devices.
9. controlling real-time alerts and updates to keep consumers informed and involved.

10. Following privacy and data protection rules across the globe.

1.4 Project Description

Advanced machine learning and image recognition technologies will be combined in the solution we propose. It will include:

- a backend using FastAI for strong price prediction and image recognition.
- Users with a frontend interface can manually enter car details or upload car photos to obtain price predictions and model information.
- To improve the accuracy and efficiency of the system, a preprocessing stage using YOLO analyzes if pictures given by users really include cars.

1.5 Scope

- developing an extensive pricing plan that changes in line with trends and real-time market data.
- Creating an image recognition module that can accurately identify car models from user-uploaded images.
- building a module for image recognition able to accurately recognizing vehicle models submitted by users.
- Regular platform updates reflecting the most recent car pricing and model data help ensure that the system stays accurate and active.

Chapter 2

Background

The project automatically extracts vehicle information from images and uses that data to predict a fair market price for used cars. The system employs image recognition techniques to determine a vehicle's make, model, and production year. This information is then fed into a regression model to estimate the car's price. Such a system benefits car buyers, sellers, dealerships, and automotive service providers, while also addressing economic and environmental concerns in the automotive market.

Key points include:

- **Wider Context:** The automotive market is shifting toward used vehicles due to economic pressures and environmental concerns.
- **Identified Problem:** Existing solutions often do not integrate image-based recognition with price prediction or lack sufficient accuracy in real-world conditions.
- **Stakeholders:** Car buyers, sellers, dealerships, and automotive service providers.
- **Relevant Theories:** Deep learning, computer vision, transfer learning, and regression analysis.
- **Constraints:** Variability in image quality, differences in vehicle appearance, and the need for rapid processing.
- **Existing Solutions:** Previous work in vehicle recognition and price estimation shows promise but is limited when dealing with complex, real-world data.

2.1 Literature Review

The summaries below describe research studies relevant to vehicle recognition and price prediction. Each summary addresses the study's motivation, the problem it tackles, the contributions made, the main results, and its relevance to our project.

General Studies

label=[1]

1. **Used Car Price Prediction using Machine Learning: A Case Study [1]**

Motivation: To provide a reliable method for predicting used car prices.

Problem: Car prices depend on many factors (mileage, age, condition) which complicate predictions.

Contribution: A regression model using gradient boosting was developed.

Results: The model achieved high prediction accuracy.

Relevance: This work supports the price prediction module of our system.

2. **DeepCar 5.0: Vehicle Make and Model Recognition Under Challenging Conditions [2]**

Motivation: To improve vehicle recognition performance under poor image conditions.

Problem: Many systems fail when images are noisy or only partially captured.

Contribution: A multi-agent framework was proposed that focuses on key parts of the vehicle.

Results: The system reported accuracies between 92% and 96%.

Relevance: The techniques used here are applicable to our image processing module.

3. **An AI-based Visual Attention Model for Vehicle Make and Model Recognition [3]**

Motivation: To enhance focus on the most important regions of vehicle images.

Problem: Standard CNNs may overlook small but crucial differences between similar vehicles.

Contribution: A recurrent attention unit (RAU) was introduced to improve CNN performance.

Results: Accuracy improvements were shown on benchmarks like the Stanford Cars dataset.

Relevance: This study informs our design of attention mechanisms for feature extraction.

4. **Vehicle Make and Model Recognition using Deep Transfer Learning and Support Vector Machines [4]**

Motivation: To accurately differentiate between very similar vehicle models.

Problem: Fine-grained vehicle recognition is challenging due to high similarity among classes.

Contribution: A pre-trained VGG16 network was combined with genetic algorithms for feature selection and SVM for classification.

Results: The method achieved over 98% accuracy.

Relevance: This hybrid approach demonstrates effective vehicle recognition.

5. **Prediction of Prices Car Price Prediction with Machine Learning [5]**

Motivation: To develop an accurate predictive model for used car pricing.

Problem: Multiple features influence car prices, making prediction complex.

Contribution: Several machine learning algorithms were evaluated to select the best model.

Results: The chosen model outperformed conventional methods.

Relevance: This work underpins our use of advanced regression techniques.

6. **Multiple Linear Regression Algorithm-based Car Price Prediction [6]**

Motivation: To test if simple linear regression can predict car prices effectively.

Problem: The influence of many factors complicates simple regression models.

Contribution: A comparison of various regression models was conducted.

Results: Multiple linear regression achieved approximately 84.5% accuracy.

Relevance: This study establishes a baseline for price prediction.

7. **Are Used Cars More Sustainable? Price Prediction Based on Linear Regression [7]**

Motivation: To connect vehicle price prediction with sustainability concerns.

Problem: Integrating economic and environmental factors into one model is challenging.

Contribution: A linear regression model was developed and evaluated using RMSE and other metrics.

Results: The model achieved a low RMSE of 1.43, indicating robust performance.

Relevance: This research links price prediction to sustainability, a key aspect of our project.

8. **Last Decade in Vehicle Detection and Classification: A Comprehensive Survey [8]**

Motivation: To review progress in vehicle detection and classification over the past decade.

Problem: The large number of existing methods makes it hard to choose the best approach.

Contribution: A thorough survey of various techniques, trends, and gaps was presented.

Results: A comprehensive overview of the state-of-the-art was provided.

Relevance: This survey offers context and identifies opportunities for improvement.

9. **Real-time Vehicle Make and Model Recognition System [9]**

Motivation: To create a system that recognizes vehicles in real time.

Problem: Changing conditions in dynamic environments make real-time recognition challenging.

Contribution: A system based on random forest classification was developed and optimized for speed.

Results: Promising real-time performance was demonstrated.

Relevance: Real-time recognition is critical for applications such as traffic management.

10. **Structural Description Model for Vehicle Feature Recognition [10]**

Motivation: To improve feature extraction by incorporating structural information of vehicles.

Problem: Traditional methods may miss important shape and layout details.

Contribution: A structural description model was proposed to capture the vehicle's form.

Results: The model outperformed earlier methods in feature recognition tasks.

Relevance: Enhanced feature extraction is essential for both vehicle recognition and price prediction.

11. Multi-class and Single-class Classification Approaches to Vehicle Model Recognition from Images [11]

Motivation: To compare different classification strategies for vehicle recognition.

Problem: It is unclear whether multi-class or single-class classification yields better performance.

Contribution: A comparison was made using a diverse dataset of vehicle images.

Results: The study provided insights into the strengths and weaknesses of each approach.

Relevance: This work informs our selection of an effective classification strategy.

Deep CNN-based Vehicle Make and Model Recognition Methods

1. Sochor et al. (2016) [12]

Motivation: Improve recognition by incorporating 3D structural information.

Problem: Standard CNNs may not capture important 3D shape details from 2D images.

Contribution: Combined CNN features with 3D bounding box data and a rasterized low-resolution shape.

Results: Achieved 85% accuracy on easy cases, 82.7% on medium cases, and 76.8% on hard cases using the BoxCars dataset.

Relevance: This approach shows that adding spatial 3D information can enhance recognition.

2. Ren et al. (2016) [13]

Motivation: Build a simple yet robust CNN for vehicle recognition.

Problem: High recognition accuracy is difficult to achieve with a limited

training set.

Contribution: Trained a CNN on 29,847 images from the CompCars dataset.

Results: Reached an accuracy of 98.7%.

Relevance: Demonstrates that a straightforward CNN can achieve excellent performance.

3. **Dehghan et al. (2017) [14]**

Motivation: Enable view-independent vehicle recognition.

Problem: Variations in viewing angles lower recognition accuracy.

Contribution: Developed a system using Sighthound's deep networks that handles multiple viewing conditions.

Results: Achieved 93% accuracy on easy, 86.52% on medium, and 80.17% on hard cases.

Relevance: Highlights the importance of robustness to different perspectives.

4. **Tafazzoli et al. (2017) [15]**

Motivation: Leverage established deep CNN models for vehicle recognition.

Problem: A limited number of images in some classes reduces accuracy.

Contribution: Applied ResNet50 and VGG16 on the VMNR-db-3036 dataset.

Results: Achieved 92.90% accuracy (top-5) and 51.76% accuracy (top-1).

Relevance: Indicates challenges with imbalanced data in recognition tasks.

5. **Liu et al. (2017) [16]**

Motivation: Assess the performance of various deep models for fine-grained vehicle classification.

Problem: Sparse training data affect the performance of some models.

Contribution: Compared VGG16, CaffeNet, and GoogleNet on 16,185 images covering 196 classes.

Results: Achieved 95.1% accuracy (top-5) and 80% accuracy (top-1).

Relevance: Helps in selecting a suitable deep learning architecture.

6. **Zwemer et al. (2017) [17]**

Motivation: Use an established CNN (AlexNet) for vehicle recognition from frontal images.

Problem: Recognition from a single frontal view is challenging.

Contribution: Trained an AlexNet-based model on a dataset with 500 vehicle models.

Results: Achieved 98% accuracy (top-1).

Relevance: Shows that older CNN architectures can still be effective.

7. **Dai et al. (2017) [18]**

Motivation: Combine object detection with classification for improved results.

Problem: The system must both detect and recognize vehicles accurately.

Contribution: Used ResNet and VGG16 on the Stanford Cars dataset for joint detection and classification.

Results: Reported an accuracy of 93.1%.

Relevance: Supports the idea of integrating detection and recognition tasks.

8. **Lu and Huang (2018) [19]**

Motivation: Enhance recognition accuracy through a multi-stage process.

Problem: Single-stage models may not capture both general and detailed features.

Contribution: Implemented a two-stage system on 12,238 images covering 400 models from 58 makes.

Results: Achieved 95.01% accuracy.

Relevance: Illustrates the advantages of dividing the recognition process.

9. **Tian et al. (2018) [20]**

Motivation: Detect subtle differences among similar vehicle models.

Problem: Standard CNNs may miss minor discriminative details.

Contribution: Proposed an ID-CNN using selective multi-convolutional region features.

Results: Achieved 91.8% accuracy on the Stanford Cars dataset and 96.2% on the CompCars dataset.

Relevance: Confirms that focusing on key regions enhances recognition performance.

2.2 Summary

The above-shown studies show significant improvement in vehicle recognition and pricing prediction. General research ([1]–[11]) indicates that, considering several features, machine learning and regression techniques may reasonably estimate used car values. Using

advanced convolutional neural network architectures—especially when combined with attention mechanisms, multiple phase processing, and combining techniques—the deep CNN-based algorithms ([13]–[21]) show that high accuracy in recognizing car make and model may be attained. These results give the present study, which combines image-based vehicle identification with a regression model to give real-time, dependable pricing information for used cars, a strong basis.

2.3 Comparison with Proposed Project

In Egypt, it is difficult for people to determine the appropriate price for used cars since sellers sometimes post vehicles at either too high or too low simply to draw attention; prices change frequently. This makes it challenging for sellers as well as people to determine what a reasonable price is. Combining two important components in one system—recognizing car information from images and estimating car prices—our project deals with this issue.

Our system works in two steps. First, it uses image recognition to find out the car’s make, model, and production year. We use a DenseNet201-based model that can correctly identify these details about 94% of the time. This helps users who do not know much about cars because they can simply upload a photo, and the system will tell them the basic details of the car.

The second step is price prediction. Once the car’s details are known, our system uses a regression model based on XGBoost to estimate a fair price for the car. This model has shown an accuracy of 97%, meaning it can give a very good price estimate based on the car’s features, such as its age, mileage, and condition.

The main differences between our project and other solutions are:

- **Combined Functionality:** Instead of only recognizing cars or only predicting prices, our system does both. Users can just upload an image and get the car’s details and its estimated price.
- **High Accuracy Models:** We use a modern deep learning model (DenseNet201) for recognizing car details and a strong regression model (XGBoost) for predicting prices. Both models work well even when images are not perfect.
- **Easy to Use:** Our user interface is designed to be simple and clear. Users can upload photos or enter details, and the system will show the results in an easy-to-understand format.

- **Fast and Scalable:** The system is built to work quickly with many users and to update the information in real time as market prices change.

In short, our project creates a simple and clear solution for the Egyptian used car market. By combining car recognition from images with accurate price prediction, the system helps buyers, sellers, and dealerships make better decisions. It gives trustworthy, real-time information, making the process of buying or selling a car more fair and easier for everyone.

2.4 Screen Shots from Previous Systems (if needed)

Below are some screenshots from previous systems to give context and inspiration for our project.



Figure 2.1: Example Screenshot from a Previous System

2.5 Project Management and Deliverable

2.5.1 Tasks and Time Plan

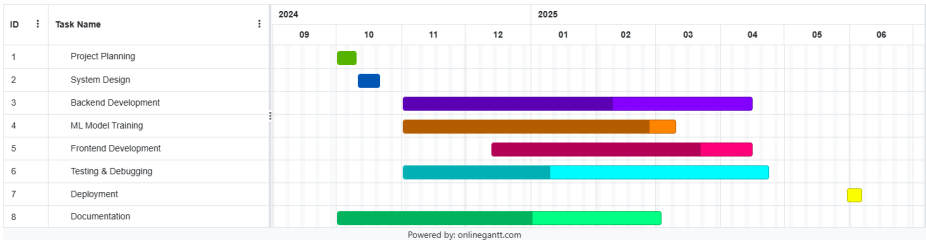


Figure 2.2: Gantt Chart for Project Plan

2.5.2 Budget and Resource Costs

Total initial budget: 15,000 EGP

Chapter 3

Specification - (SRS)

3.1 Introduction

3.1.1 Purpose of this Document

The requirements for our project are clarified in this document. The project is a software program designed to assist sellers and buyers in determining reasonable used car pricing. From an image, it automatically recognizes the brand, model, and manufacturing year of a car. A machine learning algorithm using this data estimates the market price of the car. This document is written for users, customers, system engineers, and developers.

3.1.2 Scope of this Document

This document covers the full requirements for the car price prediction and vehicle recognition system. It explains what the system must do, how it will work, and the limits it must operate within. The requirements were gathered with input from users, customers, system engineers, and developers. The document also notes any limits such as time, cost, and the development environment.

3.1.3 Overview

The software program uses car images and details to estimate a reasonable market value. It consists mostly on two sections:

- **Image Recognition:** This part automatically extracts from a car picture the make, model, and manufacturing year.

- **Price Prediction:** Using a regression model, this part estimates a reasonable price based on the information gained and additional factors including miles and CC.

The software is meant to be user-friendly even for people without cars knowledge.

3.1.4 Business Context

The program is designed for a business aiming to provide justice and openness in the used car market. In many markets, including Egypt, vehicle costs can vary significantly, and purchasers sometimes have no idea if they are paying a reasonable amount. Our method helps provide reliable and clear car values. The company wants to create confidence among sellers and buyers, reduce the purchasing and selling process, and assist a sustainable car market.

3.2 General Description

3.2.1 Product Functions

The product is a web-based application designed to let consumers get a used car's fair market pricing. If users are unsure about the car specifications, they can upload a picture; else, they can complete a form including the car's information (such as make, model, year, mileage, and condition). The system then determines the make, model, and year of the car by use of an image recognition module. A regression model predicts the price of the car once the information about it is present. The technology also lets several people create and log into their accounts. An administrator helps run the system and can view scraped data from the online car advertisements on a dashboard.

3.2.2 Similar System Information

This product is designed to work both as a stand-alone application and as a service that can be integrated into other software systems (Software as a Service). For example, the website <https://eg.hat1a2ee.com/> offers a car evaluation model that only uses a few parameters (make, model, year, and mileage) to predict prices; however, it does not work well for all cars. Another service, <https://carnet.ai/>, provides a car recognition API, but its API is not free and only returns basic information such as car make, model, year,

and some images. In contrast, our product offers both a detailed form-based input and an image recognition module, and it is designed to work well even when some information is missing.

3.2.3 User Characteristics

Dealerships, sellers, and car buyers are the primary users of this system. Many users lack knowledge in technical car details or car value. They require a basic and straightforward tool to enable them to quickly recognize the features and fair price of a car. Furthermore, used by system administrators, will be the dashboard and data scraping tools for updating the system.

3.2.4 User Problem Statement

Users face several challenges in the current used car market. Buyers and sellers often have difficulty determining a fair price because the available tools either use very limited data (such as only make, model, year, and mileage) or require manual input of detailed car information. Non-expert users may not know all the technical details of a car, which leads to inaccurate price estimates and confusion in the market.

3.2.5 User Objectives

The system is designed to assist users by:

- allowing consumers enter a basic form with vehicle specifics or upload a car picture in case specifics are unknown.
- offering an acceptable pricing estimate based on multiple factors.
- allowing users log into personal accounts to save and review their information.
- Providing tools for administrators of the system, dashboards, and web-based data scrap management.
- Offering a service that can be integrated with other platforms if needed.

3.2.6 General Constraints

The following constraints should guide system design and development:

- The system has to change pricing right away and handle a lot of data.
- It should run on common web and mobile devices.

- The image recognition module has to work with images taken under several lighting and viewpoints.
- User privileges and data security have to be guaranteed.
- Common industry standards should be used in building the system to provide simple integration as a service with other products.

3.3 Functional Requirements

Table 3.1: Functional Requirement FR-001: User Registration

Function Name	User Registration
Description	The system must allow new users to sign up by entering their name, email, phone number, and password. This creates an account so users can log in and use the system.
Criticality	High – This is a core feature for user access.
Technical Issues	Securely storing passwords and sending email verifications.
Cost and Schedule	Moderate – Involves email integration and secure authentication.
Risks	Registration may fail if the network is slow or the input is wrong; use robust validation and retry mechanisms.
Dependencies	Relies on the Login function for account access.
Pre-Condition	The system is active and ready to accept new user data.
Post-Condition	A new account is created and the user can subsequently log in.

Table 3.2: Functional Requirement FR-002: User Login

Function Name	User Login
Description	The system must authenticate users using their registered email and password, granting access only with valid credentials.
Criticality	High
Technical Issues	Secure password checking (e.g., bcrypt) and JWT token issuance.
Cost and Schedule	Moderate
Risks	Incorrect password entries or forgotten credentials; password recovery is necessary.
Dependencies	Depends on successful user registration.
Pre-Condition	User has an existing account.
Post-Condition	User is authenticated and receives a JWT token for further access.

Table 3.3: Functional Requirement FR-003: User Logout

Function Name	User Logout
Description	The system must allow users to securely log out, terminating their session and clearing authentication tokens.
Criticality	High
Technical Issues	Proper token invalidation and session clearance.
Cost and Schedule	Low
Risks	Persisting tokens might lead to unauthorized access if not cleared.
Dependencies	Depends on User Login.
Pre-Condition	User is logged in.
Post-Condition	Session is terminated and the user is redirected appropriately.

Table 3.4: Functional Requirement FR-004: User Profile Management

Function Name	User Profile Management
Description	Users must be able to view and update their profile information (name, email, phone, etc.) to maintain accurate account details.
Criticality	High
Technical Issues	Secure data update with proper validation.
Cost and Schedule	Moderate
Risks	Unauthorized modifications if access control fails.
Dependencies	Depends on User Login.
Pre-Condition	User is authenticated.
Post-Condition	Updated profile is stored in the system.

Table 3.5: Functional Requirement FR-005: Password Recovery

Function Name	Password Recovery
Description	The system should allow users to reset their password through an email-based recovery process to restore account access.
Criticality	High
Technical Issues	Secure token generation, email integration, and token expiration management.
Cost and Schedule	Moderate
Risks	Recovery process must be secure to prevent unauthorized resets.
Dependencies	Depends on User Registration and Login functions.
Pre-Condition	User has a registered email address.
Post-Condition	User is able to set a new password.

Table 3.6: Functional Requirement FR-006: Car Listing Creation

Function Name	Car Listing Creation
Description	Authenticated users must be able to create a car listing by providing details such as title, make, model, year, mileage, price, images, and description.
Criticality	High
Technical Issues	Handling file uploads and validating user input.
Cost and Schedule	Moderate
Risks	Incomplete or invalid data entry; robust validation is necessary.
Dependencies	Depends on User Authentication and Data Validation.
Pre-Condition	User is logged in.
Post-Condition	The new car listing is saved and displayed in the marketplace.

Table 3.7: Functional Requirement FR-007: Edit Car Listing

Function Name	Edit Car Listing
Description	Users must be able to modify the details of their existing car listings.
Criticality	High
Technical Issues	Consistent update mechanisms with robust validation.
Cost and Schedule	Moderate
Risks	Data inconsistency or update failures; proper error handling is required.
Dependencies	Depends on Car Listing Creation.
Pre-Condition	User is logged in and owns the listing.
Post-Condition	The listing is updated in the system.

Table 3.8: Functional Requirement FR-008: Delete Car Listing

Function Name	Delete Car Listing
Description	Users must be able to delete their car listings from the system.
Criticality	High
Technical Issues	Implement safe deletion (or soft-deletion) with confirmation prompts.
Cost and Schedule	Low
Risks	Accidental deletion; include confirmation and potential undo options.
Dependencies	Depends on Car Listing Creation.
Pre-Condition	User is logged in and owns the listing.
Post-Condition	The listing is removed (or marked inactive) in the system.

Table 3.9: Functional Requirement FR-009: View Car Listings

Function Name	View Car Listings
Description	The system must display all active car listings in a browsable format to users.
Criticality	High
Technical Issues	Efficient querying and rendering of large datasets.
Cost and Schedule	Moderate
Risks	Slow performance if data is not properly indexed.
Dependencies	Depends on Car Listing Creation.
Pre-Condition	Active listings exist in the system.
Post-Condition	Users see a list of available car listings.

Table 3.10: Functional Requirement FR-010: Browse/Search/Filter Listings

Function Name	Browse/Search/Filter Listings
Description	Users must be able to search and filter car listings based on criteria such as make, model, year, and price.
Criticality	High
Technical Issues	Fast query processing and an intuitive search UI.
Cost and Schedule	Moderate
Risks	Inaccurate or slow search results; proper indexing is essential.
Dependencies	Depends on effective data retrieval mechanisms.
Pre-Condition	User has entered search criteria.
Post-Condition	Listings matching the criteria are displayed.

Table 3.11: Functional Requirement FR-011: View Listing Details

Function Name	View Listing Details
Description	Users must be able to click on a listing to view detailed information (images, description, seller contact if permitted).
Criticality	High
Technical Issues	Responsive display and proper rendering of images and text.
Cost and Schedule	Low to Moderate
Risks	Incomplete data display if image links are broken.
Dependencies	Depends on View Car Listings.
Pre-Condition	User selects a listing from the marketplace.
Post-Condition	Detailed listing information is displayed.

Table 3.12: Functional Requirement FR-012: Car Price Prediction

Function Name	Car Price Prediction
Description	Users must be able to input car details (make, model, year, mileage, etc.) and receive a predicted price using an XGBoost model.
Criticality	High
Technical Issues	Integration of an accurate regression model and thorough input validation.
Cost and Schedule	High – Requires ongoing model training and updates.
Risks	Inaccurate predictions if the model is not updated; periodic retraining is necessary.
Dependencies	Depends on Data Validation and ML integration.
Pre-Condition	User provides complete and valid car details.
Post-Condition	The predicted car price is displayed.

Table 3.13: Functional Requirement FR-013: Car Damage Detection

Function Name	Car Damage Detection
Description	The system must allow users to upload an image of a car and perform damage detection using a YOLOv8 model.
Criticality	High
Technical Issues	High computational requirements and robust ML integration for image processing.
Cost and Schedule	High
Risks	Inaccurate damage detection if image quality is poor; preprocessing is critical.
Dependencies	Depends on the Image Upload process and ML integration.
Pre-Condition	User uploads a valid car image.
Post-Condition	Damage analysis results are displayed.

Table 3.14: Functional Requirement FR-014: Car Recognizer

Function Name	Car Recognizer
Description	The system must allow users to upload a car image to automatically extract details (make, model, year) using a DenseNet201 model.
Criticality	High
Technical Issues	Integration of deep learning model and efficient image processing.
Cost and Schedule	High
Risks	Incorrect recognition if model accuracy is insufficient; requires model updates.
Dependencies	Depends on Image Upload and ML integration.
Pre-Condition	User uploads a car image.
Post-Condition	Recognized car details are displayed and can auto-fill listing forms.

Table 3.15: Functional Requirement FR-015: Car Image Validation

Function Name	Car Image Validation
Description	The system must validate that the uploaded image shows a car and reject any image that does not meet this criterion.
Criticality	High
Technical Issues	Real-time image validation using ML techniques.
Cost and Schedule	Moderate
Risks	Poor validation may allow invalid images, leading to erroneous ML outputs.
Dependencies	Depends on the Image Upload process and ML integration.
Pre-Condition	User selects an image to upload.
Post-Condition	Only valid car images are accepted; otherwise, prompt user for re-upload.

Table 3.16: Functional Requirement FR-016: Send Message

Function Name	Send Message
Description	Users must be able to send messages to sellers regarding car listings.
Criticality	High
Technical Issues	Real-time messaging using HTTP/WebSocket and ensuring message persistence.
Cost and Schedule	Moderate
Risks	Message delivery failures; require confirmation and error handling.
Dependencies	Depends on User Login and Car Listing functions.
Pre-Condition	User is logged in and viewing a listing.
Post-Condition	The message is sent and stored in the system.

Table 3.17: Functional Requirement FR-017: Retrieve Conversation History

Function Name	Retrieve Conversation History
Description	The system must allow users to view the history of messages exchanged with other users.
Criticality	High
Technical Issues	Efficient retrieval and pagination of message records.
Cost and Schedule	Moderate
Risks	Performance issues with large volumes; optimize queries.
Dependencies	Depends on the messaging system.
Pre-Condition	User has existing conversations.
Post-Condition	Conversation history is displayed to the user.

Table 3.18: Functional Requirement FR-018: Mark Message as Read

Function Name	Mark Message as Read
Description	The system should update the status of messages to "read" once they are viewed by the recipient.
Criticality	Medium
Technical Issues	Implement reliable status flag updates.
Cost and Schedule	Low
Risks	Incorrect status updates if not properly managed.
Dependencies	Depends on the messaging system.
Pre-Condition	Message has been delivered.
Post-Condition	The message status is updated to "read".

Table 3.19: Functional Requirement FR-019: View System Statistics (Admin Dashboard)

Function Name	View System Statistics
Description	The admin must be able to view key statistics (total users, listings, messages) on a dashboard.
Criticality	High
Technical Issues	Efficient data aggregation and real-time display.
Cost and Schedule	Moderate
Risks	Outdated statistics if data is not refreshed periodically.
Dependencies	Depends on data collection and database updates.
Pre-Condition	Admin is logged in and data exists.
Post-Condition	Up-to-date statistics are displayed on the dashboard.

Table 3.20: Functional Requirement FR-020: Manage Users (Admin Dashboard)

Function Name	Manage Users
Description	The admin must be able to view, update roles, and deactivate/reactivate user accounts.
Criticality	High
Technical Issues	Secure role-based access control and update mechanisms with audit logging.
Cost and Schedule	Moderate
Risks	Unauthorized changes if not properly secured; audits are required.
Dependencies	Depends on User Authentication and Profile Management.
Pre-Condition	Admin is logged in.
Post-Condition	User accounts are updated as per admin actions.

Table 3.21: Functional Requirement FR-021: Manage Car Listings (Admin Dashboard)

Function Name	Manage Car Listings
Description	The admin must be able to view, edit, and delete or deactivate any car listing.
Criticality	High
Technical Issues	Secure listing modification and deletion with proper audit trails.
Cost and Schedule	Moderate
Risks	Accidental deletion; include confirmation prompts and logs.
Dependencies	Depends on Car Listing functions.
Pre-Condition	Admin is logged in.
Post-Condition	Listings are updated or removed as per admin actions.

Table 3.22: Functional Requirement FR-022: Data Visualization (Admin Dashboard)

Function Name	Data Visualization
Description	The system must generate visual charts (using Recharts) to display market trends and analytics.
Criticality	High
Technical Issues	Efficient data aggregation and responsive chart rendering.
Cost and Schedule	Moderate
Risks	Incorrect visualization if data is outdated; ensure real-time updates.
Dependencies	Depends on data retrieval and database functions.
Pre-Condition	Admin is logged in and data is available.
Post-Condition	Visual analytics are clearly displayed on the dashboard.

Table 3.23: Functional Requirement FR-023: Highest Car Make Analysis (Admin Data Analysis)

Function Name	Highest Car Make Analysis
Description	The system should analyze car listings to determine and display the most frequently listed car make.
Criticality	High
Technical Issues	Fast aggregation and filtering algorithms.
Cost and Schedule	Moderate
Risks	Inaccurate results if data is not refreshed regularly.
Dependencies	Depends on effective data scraping and collection.
Pre-Condition	Sufficient car listing data exists.
Post-Condition	The most frequent car make is displayed to the admin.

Table 3.24: Functional Requirement FR-024: Highest Car Model Analysis (Admin Data Analysis)

Function Name	Highest Car Model Analysis
Description	The system should analyze car listings to determine and display the most frequently listed car model.
Criticality	High
Technical Issues	Efficient sorting and counting algorithms.
Cost and Schedule	Moderate
Risks	Data inaccuracies if not updated; periodic data refresh is needed.
Dependencies	Depends on effective data scraping.
Pre-Condition	Adequate car listing data is available.
Post-Condition	The most common car model is displayed to the admin.

Table 3.25: Functional Requirement FR-025: Fuel Type Distribution (Admin Dashboard)

Function Name	Fuel Type Distribution
Description	The system must display a breakdown of car listings by fuel type (e.g., Petrol, Diesel).
Criticality	High
Technical Issues	Accurate filtering and classification of listings.
Cost and Schedule	Moderate
Risks	Inconsistent data could lead to misclassification; ensure standardization.
Dependencies	Depends on reliable data collection and validation.
Pre-Condition	Listings include fuel type information.
Post-Condition	Fuel type distribution is clearly visualized on the dashboard.

Table 3.26: Functional Requirement FR-026: Market Trend Analysis (Admin Dashboard)

Function Name	Market Trend Analysis
Description	The system must analyze historical car listing data to provide trends (e.g., yearly price trends).
Criticality	High
Technical Issues	Efficient aggregation of historical data and trend visualization.
Cost and Schedule	Moderate
Risks	Trends may be inaccurate if historical data is incomplete; ensure continuous data collection.
Dependencies	Depends on continuous data scraping and collection.
Pre-Condition	Historical data is available.
Post-Condition	Market trends are displayed on the admin dashboard.

Table 3.27: Functional Requirement FR-027: Data Scraping and Scheduling (Admin)

Function Name	Data Scraping and Scheduling
Description	The admin must be able to initiate and schedule web scraping tasks to update external car data.
Criticality	High
Technical Issues	Dynamic scheduling and robust scraping scripts that handle diverse website formats.
Cost and Schedule	High – Ongoing maintenance is required.
Risks	Changes in external websites may break the scraper; regular updates are necessary.
Dependencies	Depends on network access and scraping service integration.
Pre-Condition	Admin is logged in and system configuration is active.
Post-Condition	New car data is scraped and integrated into the database.

Table 3.28: Functional Requirement FR-028: Error Handling and Logging

Function Name	Error Handling and Logging
Description	The system must display clear error messages for invalid inputs and log all system activities for debugging and auditing.
Criticality	High
Technical Issues	Robust logging mechanisms and user-friendly error reporting.
Cost and Schedule	Low to Moderate
Risks	Insufficient logging may hinder troubleshooting; ensure comprehensive log coverage.
Dependencies	Depends on all input and processing functions.
Pre-Condition	System is operational.
Post-Condition	Errors are displayed and events are logged appropriately.

Table 3.29: Functional Requirement FR-029: Data Validation for Car Details

Function Name	Data Validation for Car Details
Description	The system must validate that all required fields (make, model, year, mileage, etc.) for a car listing are correctly filled before submission.
Criticality	High
Technical Issues	Robust validation rules with real-time feedback.
Cost and Schedule	Moderate
Risks	Invalid data may lead to inaccurate predictions; strict validation is needed.
Dependencies	Depends on Car Listing Creation.
Pre-Condition	User has entered car details in the listing form.
Post-Condition	Only validated data is accepted and processed.

Table 3.30: Functional Requirement FR-030: Responsive UI/UX Design

Function Name	Responsive UI/UX Design
Description	The system must provide a user-friendly, responsive interface that adapts seamlessly to desktop and mobile devices.
Criticality	High
Technical Issues	Implementation using responsive frameworks (e.g., Material UI) and thorough testing.
Cost and Schedule	Moderate
Risks	Poor design could lead to usability issues; extensive testing is required.
Dependencies	Depends on Frontend Development.
Pre-Condition	Users access the system via web browsers.
Post-Condition	The interface renders correctly on all devices.

Table 3.31: Functional Requirement FR-031: Account Deactivation by User

Function Name	Account Deactivation
Description	The system should allow users to temporarily deactivate their account without data loss.
Criticality	Medium
Technical Issues	Secure process to disable the account while preserving data for reactivation.
Cost and Schedule	Low
Risks	The reactivation process must be secure and user-friendly.
Dependencies	Depends on User Profile Management.
Pre-Condition	User is logged in.
Post-Condition	The account is marked as inactive.

Table 3.32: Functional Requirement FR-032: Car Listing Image Management

Function Name	Car Listing Image Management
Description	The system should allow users to upload, update, and delete multiple images for a car listing.
Criticality	Medium
Technical Issues	Efficient file upload, storage, and retrieval; handling large files with compression if needed.
Cost and Schedule	Moderate
Risks	Large image sizes may slow performance; implement file size limits and compression.
Dependencies	Depends on Car Listing Creation and Data Storage systems.
Pre-Condition	User is creating or editing a car listing.
Post-Condition	Multiple images are correctly associated with the listing and displayed.

3.4 Interface Requirements

3.4.1 User Interfaces

The product interacts with the user through a web-based application, enabling users to interact with several capabilities like image recognition and car pricing estimation. The user interfaces with the system are outlined here:

- **Web Frontend:** React.js is used building the web application to offer a dynamic and responsive user experience. React components manage user interactions including form inputs, picture uploads, and result display.
- **Material-UI Components:** Material-UI (MUI) pre-designed UI components—text fields, buttons, sliders—are used on the interface. These elements ensure a constant and visually appealing design all through the application.
- **Charts (Recharts):** The system generates several charts to show market patterns like the most often used vehicle make, model, fuel type distribution, and price range. Recharts, an open-source React charting application, helped generate these charts.
- **Car Price Prediction Form:** Users enter car information including make, model, year, kilometers, fuel type, transmission, and body type on a form. When

submitted, the system analyzes the information and generates an estimated vehicle price. The form is checked to guarantee correct completion of all fields; the outcome shows the expected price.

- **Image Recognition Interface:** Users who might not know the car specs can upload a picture of it. The algorithm finds the car's make, model, and year by using an image recognition model. The details are shown on the user interface; the price of the car will be estimated depending on the found information.
- **Error Handling:** If the user inputs incorrect information or uploads an invalid image, the system will provide an error notice asking the user to fix the input. Should an invalid car image be provided, for instance, the user will be requested to upload an image showing a car.
- **Responsive Design:** Designed to be totally responsive, the user interface offers an error-free interface on many platforms, including desktop computers, tablets, and smartphones.

Overall User Flow:

1. Users log on using email and a password on their accounts.
2. On the primary dashboard, users may choose to manually enter vehicle information to estimate the price. Upload a picture of the car to automatically identify and predict prices.
3. Results—including the make, model, year, and expected price of the car—are shown on one page.
4. Admin users can view a separate dashboard allowing them to scrape new data and examine car market trends.

focusing on simplicity of use, the user interface offers quick feedback and well defined directions. Interactive elements help the user all through the procedure, therefore enabling even for non-technical users' navigation.

3.4.1.1 GUI

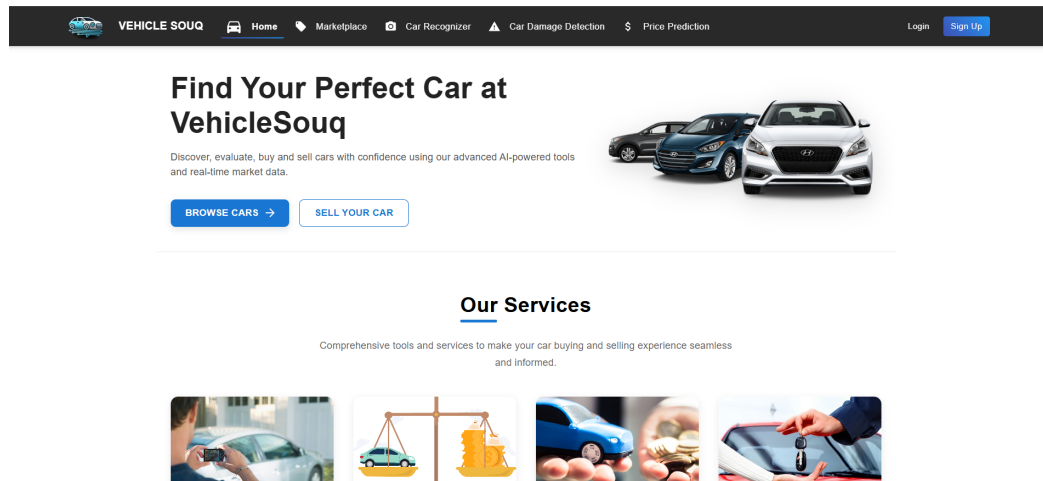


Figure 3.1: Home Page

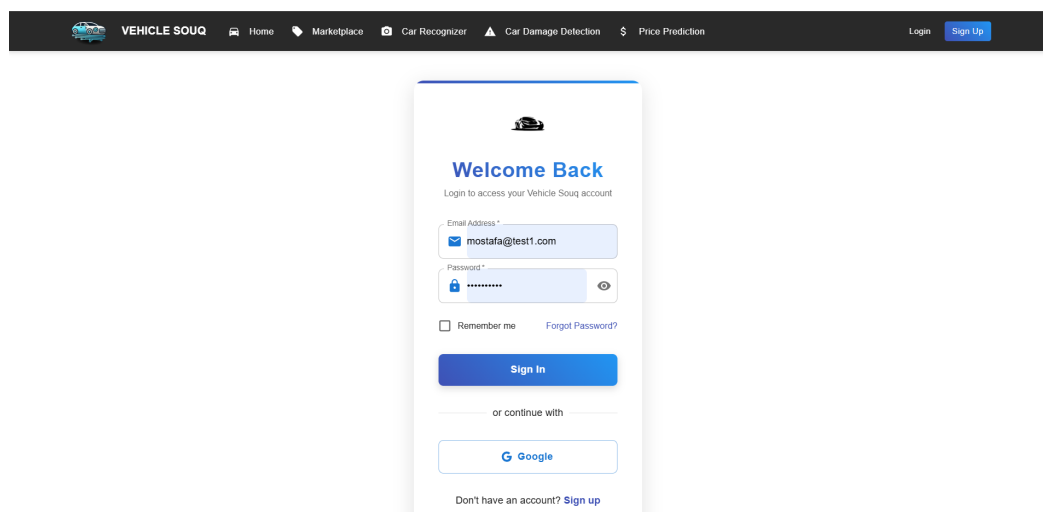


Figure 3.2: Registration Page

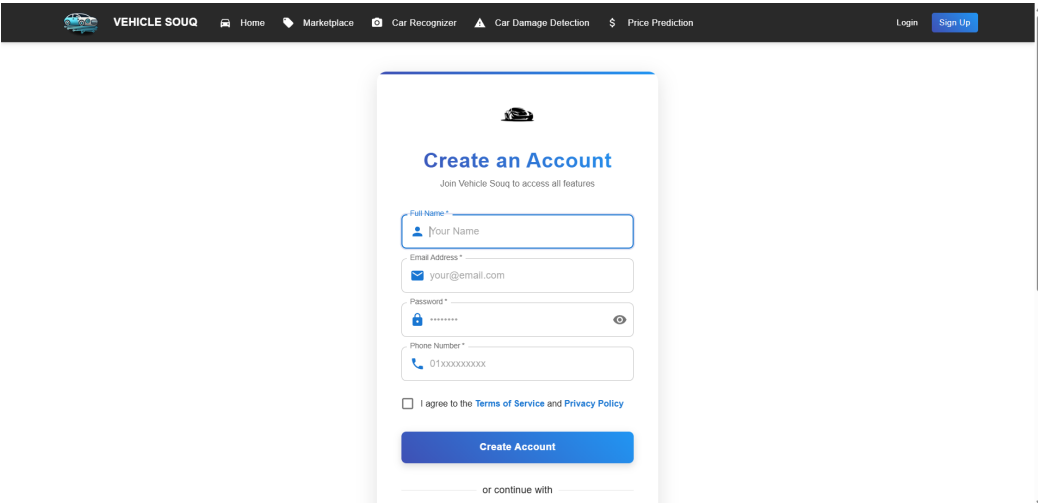


Figure 3.3: Login In

3.4.1.2 CLI

There is not a command-line interface (CLI) implemented by this system. Every contact with the system happens via the web-based graphical user interface.

3.4.1.3 API

The public application programming interface functions of the system are detailed in this part. The FastAPI backend serves all endpoints, which are reachable at `http://localhost:8000`.

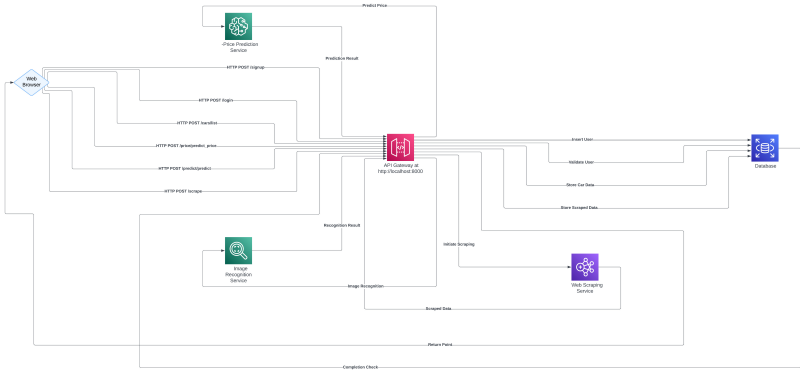


Figure 3.4: API Gateway

3.4.1.3.1 1. User Registration

- **Endpoint:** `http://localhost:8000/signup`
Method: POST
- **Arguments:** A JSON object with the following keys:
 - **name** (string): The full name of the user.
 - **email** (string): The user's email address.
 - **phone** (string): The user's phone number.
 - **password** (string): The user's password.
- **Return:** A JSON object containing a success message and the new `user_id`.
- **Example Invocation:**
POST `http://localhost:8000/signup`
Content-Type: `application/json`

```
{  
  "name": " Mostafa Hisham",  
  "email": "Mostafa@example.com",  
  "phone": "01001203311",  
  "password": "Pass12345678"  
}
```
- **Interactions:** This function must be called before logging in. Its output (the user ID) is used in the authentication process.

3.4.1.3.2 2. User Login

- **Endpoint:** `http://localhost:8000/login`
Method: POST
- **Arguments:** A JSON object with:
 - **email** (string): The user's email.
 - **password** (string): The user's password.
- **Return:** A JSON object with a success message, the user identifier, and the user role (e.g., admin or customer).
- **Example Invocation:**
POST `http://localhost:8000/login`
Content-Type: `application/json`

```
{  
  "email": "Mostafa@example.com",
```

```
    "password": "Pass12345678"
  }
```

- **Interactions:** Applied after registration to get an authentication token and access controlled endpoints.

3.4.1.3.3 3. Upload Car Listing

- **Endpoint:** `http://localhost:8000/cars/list`
Method: POST
- **Arguments:** A `FormData` object containing:
 - Car details (e.g., title, make, model, year, body type, fuel type, CC, location, kilometers, transmission type, color, condition, chat option, price, description).
 - One or more image files.

- **Return:** A JSON object with a success message and the new car listing ID.

- **Example Invocation:**

```
POST http://localhost:8000/cars/list
Content-Type: multipart/form-data
FormData {
  title: "Used BMW X5",
  make: "BMW",
  model: "X5",
  year: 2018,
  bodyType: "SUV",
  fuelType: "Benzine",
  CC: 3000,
  location: "Cairo",
  kilometers: 50000,
  transmissionType: "Automatic",
  color: "Black",
  condition: "Good",
  chatOption: "Enabled",
  price: 750000,
  description: "Well "
  file: <binary image data>
}
```

- **Interactions:** Working with the image storage module, this endpoint saves car listings into the database.

3.4.1.3.4 4. Car Price Prediction

- **Endpoint:** `http://localhost:8000/price/predictprice`
Method: *POST*
- **Arguments:** A JSON object with keys: Make, Model, BodyType, Color, Kilometer, Year, FuelType, TransmissionType, CC, location, listBy.
- **Return:** A JSON object with the predicted car price (a float).
- **Example Invocation:**
POST `http://localhost:8000/price/predict_price`
Content-Type: `application/json`

```
{
  "Make": "Toyota",
  "Model": "Corolla",
  "BodyType": "Sedan",
  "Color": "White",
  "Kilometers": 120000,
  "Year": 2015,
  "FuelType": "Benzine",
  "TransmissionType": "Automatic",
  "CC": 1800,
  "location": "Cairo",
  "listBy": "individual"
}
```
- **Interactions:** Calculates the price using pre-trained regression model, scaler, and label encoder.

3.4.1.3.5 5. YOLO Car Image Check

- **Endpoint:** `http://localhost:8000/yolo/checkcar`
Method: *POST*
- **Arguments:** A file upload containing an image.
- **Return:** A JSON object with a boolean value `car_detected` that indicates if a car is present in the image.
- **Example Invocation:**
POST `http://localhost:8000/yolo/check_car`
Content-Type: `multipart/form-data`
File: `car.jpg`

- **Interactions:** Before they are handled for additional image recognition, this endpoint confirms uploaded images.

3.4.1.3.6 6. Data Scraping

- **Endpoint:** `http://localhost:8000/scrape`
Method: POST
- **Arguments:** A JSON object with keys: `main_url`, `pages`, `fuel_type`, `transmission_type`
- **Return:** A JSON object containing a success message, scraped data, and a CSV file path.
- **Example Invocation:**
POST `http://localhost:8000/scrape`
Content-Type: `application/json`

```
{
  "main_url": "https://eg.hatla2ee.com/en/car/search",
  "pages": 5,
  "fuel_type": "Diesel",
  "transmission_type": "Automatic"
}
```

- **Interactions:** Scrapes car listing data from external websites to keep the database updated.

3.4.1.3.7 7. Data Retrieval (All Data)

- **Endpoint:** `http://localhost:8000/data/all`
Method: GET
- **Arguments:** None.
- **Return:** A JSON array of all car listing records from the database.
- **Example Invocation:**
GET `http://localhost:8000/data/all`

- **Interactions:** Used by the dashboard for data visualization and reporting.

3.4.1.3.8 8. Download CSV of Scraped Data

- **Endpoint:** `http://localhost:8000/scrape/download_csv`
Method: *GET*

- **Arguments:** None.
- **Return:** A CSV file containing scraped car listings.
- **Example Invocation:**
GET http://localhost:8000/scrape/download_csv
- **Interactions:** Allows admins to download data for offline analysis.

3.4.1.3.9 9. Price Prediction with FastAPI

- **Endpoint:** http://localhost:8000/price/predict_{price}
Method: *POST*
- **Arguments:** JSON object with car details, as described in FR-005.
- **Return:** JSON object with a key predicted_price that holds the calculated price.
- **Example Invocation:**
POST http://localhost:8000/price/predict_price
Content-Type: application/json

```
{
  "Make": "Toyota",
  "Model": "Corolla",
  "BodyType": "Sedan",
  "Color": "White",
  "Kilometers": 120000,
  "Year": 2015,
  "FuelType": "Benzine",
  "TransmissionType": "Automatic",
  "CC": 1800,
  "location": "Cairo",
  "listBy": "individual"
}
```
- **Interactions:** Integrates with the regression model and machine learning modules.

3.4.1.3.10 10. Car Image Validation for Upload

- **Endpoint:** http://localhost:8000/yolo/check_{car}
Method: *POST*
- **Arguments:** A file upload (image).

- **Return:** JSON object with a boolean `car_detected` indicating if the image shows a car.
- **Example Invocation:**
POST `http://localhost:8000/yolo/check_car`
Content-Type: `multipart/form-data`
File: `car.jpg`
- **Interactions:** Ensures that only valid car images are processed by the image recognition module.

3.4.1.3.11 11. Data Visualization for Market Trends

- **Endpoint:** `http://localhost:8000/data/make, /data/model, /data/fuel, /data/yearly`
Method: GET
- **Arguments:** None.
- **Return:** JSON arrays for each request containing statistical data on car makes, models, fuel types, and yearly trends.
- **Example Invocation:**
GET `http://localhost:8000/data/make`
GET `http://localhost:8000/data/model`
GET `http://localhost:8000/data/fuel`
GET `http://localhost:8000/data/yearly`
- **Interactions:** These endpoints feed data to the front-end dashboard for visualization using Recharts.

3.4.1.3.12 12. Car Listing Data Retrieval for Dashboard

- **Endpoint:** `http://localhost:8000/data/random`
Method: GET
- **Arguments:** None.
- **Return:** A JSON array containing random car listing records.
- **Example Invocation:**
GET `http://localhost:8000/data/random`
- **Interactions:** Supports the dashboard's need for sample data in visual reports.

3.4.1.3.13 13. Car Specifications Retrieval

- **Endpoint:** `http://localhost:8000/car/id/{car_id}` and `http://localhost:8000/car/name/{car_name}`
Method: GET
- **Arguments:** Path parameter `car_id` (string) or `car_name` (string).
- **Return:** A JSON object containing detailed car specifications.
- **Example Invocation:**
GET `http://localhost:8000/car/id/6123abc...`
GET `http://localhost:8000/car/name/"Toyota Corolla"`
- **Interactions:** Provides detailed car specs used by the image recognition and price prediction modules.

3.4.1.3.14 14. Image Recognition for Car Identification

- **Endpoint:** `http://localhost:8000/predict`
Method: POST
- **Arguments:** A file upload (image).
- **Return:** A JSON object with keys `prediction` (the car name) and `probability` (the confidence score).
- **Example Invocation:**
POST `http://localhost:8000/predict`
Content-Type: `multipart/form-data`
File: `car.jpg`
- **Interactions:** Uses the FastAI image recognition model (DenseNet201) to identify the car and set default values for further processing.

3.4.1.3.15 15. Scrape Car Listings from External Websites

- **Endpoint:** `http://localhost:8000/scrape`
Method: POST
- **Arguments:** A JSON object with parameters: `main_url`, `pages`, `fuel_type`, `transmission_type`.
- **Return:** A JSON object containing a message, the scraped data, and the CSV file path.
- **Example Invocation:**

```
POST http://localhost:8000/scrape
Content-Type: application/json

{
  "main_url": "https://eg.hatla2ee.com/en/car/search",
  "pages": 5,
  "fuel_type": "Gas",
  "transmission_type": "Automatic"
}
```

- **Interactions:** Updating market trends depends on this endpoint, which also provides fresh data into the car database.

3.4.1.4 Diagnostics or ROM

3.4.1.4.1 Console Logs: The system prints important messages to the console. When you run the application (for example, with `uvicorn`), you can see error messages, warnings, and other useful information directly in the terminal.

3.4.1.4.2 Log Files: The system also writes logs to a file named `app.log`. This file keeps a record of events and errors, so you can check it later to see what happened if there is an issue.

3.4.1.4.3 Request Logging: Middleware in the application recognizes every HTTP request and response code. When you have to find out what went wrong, this helps monitor all activities.

3.4.1.4.4 Error Reporting: When an error occurs, the system records a clear error message together with a clear problem description. These error specifics enable fast problem fixing by developers.

3.4.1.4.5 Debug Mode: During development, you can run the application in debug mode (for example, using `uvicorn --reload`). In this mode, the system shows real-time error messages and automatically reloads when code changes, which makes it easier to find and correct problems.

These tools taken together enable you to check the system, fix issues, and guarantee that everything is working as it should.

3.4.2 Hardware Interfaces

Standard hardware platforms are intended for running the system. It requires no specific tools. The frontend is reachable via current web browsers on PCs, laptops, and mobile devices; the backend can be housed on a standard server or a cloud for example.

3.4.3 Communications Interfaces

The system uses standard network protocols to enable communication between the client and the server. All data is sent over HTTP using RESTful APIs. For example, backend endpoints such as `http://localhost:8000/login` for user login and `http://localhost:8000/price/prediction` for car price prediction are used. The frontend, built with React, communicates with the backend over the internet. The system also supports Cross-Origin Resource Sharing (CORS) so that requests from approved domains are allowed. This setup ensures that data is transmitted securely and efficiently between users' devices and the server.

3.4.4 Software Interfaces

To do its tasks, the system together uses many other software components. These contain:

- **Database Interface:** Store user data, car ads, and other information on a MongoDB database the backend links to. A database driver controls reading and writing data in order to handle this connection.
- **Third-Party Libraries:** The project requires libraries like React for the frontend, FastAPI for server development, and Axios for HTTP requests. These libraries assist with user interface display, management of responses.
- **External APIs:** For certain tasks, the system interacts with third parties. For example, it searches whether an uploaded picture includes a car using a YOLO-based API.
- **File System Interface:** The device writes to and reads from the file system. It locally stores uploaded images and CSV files produced using data scraping.

3.5 Performance Requirements

The system should run fast and use memory wisely. It must handle many requests at once and send responses in under 2 seconds on average. Memory usage should be kept low even during heavy use.

3.6 Design Constraints

The system must be built within certain limits to ensure it is reliable and easy to use.

3.6.1 Standards Compliance

The system must follow common standards such as using RESTful APIs and secure encryption for data.

3.6.2 Hardware Limitations

The system should work well on regular servers and on mobile devices, even those with limited processing power.

3.6.3 Other Constraints

- It must be compatible with modern web browsers.
- The design should allow easy scaling as user numbers grow.
- The system should be simple to integrate with other services if needed.

3.7 Other Non-Functional Attributes

3.7.1 Security

Table 3.33: Non-Functional Requirement NFR-001: Security

Function Name	Security
Description	The system must protect user data and prevent unauthorized access using encryption and secure login methods.
Criticality	High (Must Have)
Technical Issues	Implement secure protocols, encrypt sensitive data, and enforce strict access control.
Cost and Schedule	Moderate
Risks	Data breaches or unauthorized access if security is not maintained.
Dependencies	None
Pre-Condition	The system is running.
Post-Condition	User data is secure and protected.

3.7.2 Binary Compatibility

Table 3.34: Non-Functional Requirement NFR-002: Binary Compatibility

Function Name	Binary Compatibility
Description	The system must run on the specified hardware and operating systems without conflicts.
Criticality	Medium (Should Have)
Technical Issues	Ensure all compiled components work on target environments.
Cost and Schedule	Low
Risks	Incompatibility may lead to system errors or crashes.
Dependencies	Depends on thorough testing on each target platform.
Pre-Condition	Hardware and OS are available.
Post-Condition	The system runs without binary conflicts.

3.7.3 Reliability

Table 3.35: Non-Functional Requirement NFR-003: Reliability

Function Name	Reliability
Description	The system should work consistently without crashing or losing data.
Criticality	High (Must Have)
Technical Issues	Use robust error handling and frequent backups.
Cost and Schedule	Moderate
Risks	Frequent failures could reduce user trust.
Dependencies	Requires proper testing and maintenance routines.
Pre-Condition	The system is deployed.
Post-Condition	The system operates consistently and reliably.

3.7.4 Maintainability

Table 3.36: Non-Functional Requirement NFR-004: Maintainability

Function Name	Maintainability
Description	The system should be easy to update and fix. Clear code and documentation are required.
Criticality	High (Must Have)
Technical Issues	Follow coding standards and maintain detailed documentation.
Cost and Schedule	Moderate
Risks	Poor maintainability may lead to extended downtimes.
Dependencies	Depends on good coding practices.
Pre-Condition	The system is operational.
Post-Condition	The system can be easily maintained and updated.

3.7.5 Portability

Table 3.37: Non-Functional Requirement NFR-005: Portability

Function Name	Portability
Description	The system must work on different devices, operating systems, and browsers.
Criticality	Medium (Should Have)
Technical Issues	Use cross-platform frameworks and test on multiple platforms.
Cost and Schedule	Low to Moderate
Risks	Incompatibility may lead to poor user experience on some devices.
Dependencies	Depends on responsive design and thorough testing.
Pre-Condition	Devices and operating systems are available.
Post-Condition	The system runs smoothly on all supported platforms.

3.7.6 Extensibility

Table 3.38: Non-Functional Requirement NFR-006: Extensibility

Function Name	Extensibility
Description	The system should be designed to easily add new features or modules in the future.
Criticality	High (Must Have)
Technical Issues	Use a modular design and clear APIs for integration.
Cost and Schedule	Moderate
Risks	A rigid design may hinder future improvements.
Dependencies	Depends on initial design decisions.
Pre-Condition	The system is built using a modular architecture.
Post-Condition	New features can be added with minimal changes to existing code.

3.7.7 Re-usability

Table 3.39: Non-Functional Requirement NFR-007: Re-usability

Function Name	Re-usability
Description	Components of the system should be designed to be reused in other projects.
Criticality	Medium (Should Have)
Technical Issues	Write generic code and document it for future use.
Cost and Schedule	Low to Moderate
Risks	Poorly designed components may not be easily reused.
Dependencies	Depends on following modular design principles.
Pre-Condition	Components are built with reusability in mind.
Post-Condition	Code can be reused in other systems with little modification.

3.7.8 Application Compatibility

Table 3.40: Non-Functional Requirement NFR-008: Application Compatibility

Function Name	Application Compatibility
Description	The system should work well with related applications and data sources by using standard interfaces and data formats.
Criticality	Medium (Should Have)
Technical Issues	Use standard APIs and data formats for easier integration.
Cost and Schedule	Low
Risks	Incompatibility may lead to integration issues.
Dependencies	Depends on standard interface design.
Pre-Condition	Other applications use compatible formats and protocols.
Post-Condition	The system integrates smoothly with other software.

3.7.9 Resource Utilization

Table 3.41: Non-Functional Requirement NFR-009: Resource Utilization

Function Name	Resource Utilization
Description	The system should use memory and processing power efficiently to avoid waste.
Criticality	Medium (Should Have)
Technical Issues	Optimize code and database queries to minimize resource consumption.
Cost and Schedule	Low to Moderate
Risks	Poor resource management can degrade system performance.
Dependencies	Depends on efficient algorithms and system design.
Pre-Condition	The system operates under normal load conditions.
Post-Condition	The system runs efficiently with minimal resource waste.

3.7.10 Serviceability

Table 3.42: Non-Functional Requirement NFR-010: Serviceability

Function Name	Serviceability
Description	The system should be easy to update and repair when issues occur, with clear logs and troubleshooting guides.
Criticality	High (Must Have)
Technical Issues	Implement clear error logs, automated alerts, and comprehensive troubleshooting documentation.
Cost and Schedule	Moderate
Risks	Delays in fixing issues could negatively impact user experience.
Dependencies	Depends on robust error handling and monitoring.
Pre-Condition	A fault or error is detected.
Post-Condition	Issues are diagnosed and resolved promptly.

3.7.11 Performance

Table 3.43: Non-Functional Requirement NFR-011: Performance

Function Name	Performance
Description	The system should respond quickly to user actions, ideally within 2 seconds for most tasks.
Criticality	High (Must Have)
Technical Issues	Optimize code, queries, and employ caching strategies to improve speed.
Cost and Schedule	Moderate
Risks	Slow responses can result in user frustration and reduced system adoption.
Dependencies	Depends on efficient design and adequate hardware resources.
Pre-Condition	The system operates under nominal load.
Post-Condition	Tasks are completed and results delivered within the performance targets.

3.7.12 Scalability

Table 3.44: Non-Functional Requirement NFR-012: Scalability

Function Name	Scalability
Description	The system must handle increasing numbers of users and data without performance degradation.
Criticality	High (Must Have)
Technical Issues	Use load balancing, modular components, and scalable database architecture.
Cost and Schedule	Moderate
Risks	Without scalability, high load can slow or crash the system.
Dependencies	Depends on hardware resources and scalable design principles.
Pre-Condition	User traffic or data volume increases.
Post-Condition	The system maintains performance as load increases.

3.7.13 Usability

Table 3.45: Non-Functional Requirement NFR-013: Usability

Function Name	Usability
Description	The system should be intuitive and easy to use, with a clear interface that minimizes user effort.
Criticality	High (Must Have)
Technical Issues	Use modern UI frameworks (e.g., React, Material UI) and perform extensive user testing.
Cost and Schedule	Moderate
Risks	A confusing interface may lead to low user adoption.
Dependencies	Depends on good UI/UX design practices.
Pre-Condition	Users access the system via web browsers.
Post-Condition	Tasks are completed easily and efficiently by users.

3.7.14 Availability

Table 3.46: Non-Functional Requirement NFR-014: Availability

Function Name	Availability
Description	The system should be online and accessible at least 99.5% of the time.
Criticality	High (Must Have)
Technical Issues	Use redundant systems and backup strategies to minimize downtime.
Cost and Schedule	Moderate
Risks	Downtime can negatively affect user confidence and business operations.
Dependencies	Depends on stable network and reliable hardware.
Pre-Condition	The system is deployed.
Post-Condition	The system remains accessible for the vast majority of the time.

3.7.15 Data Integrity

Table 3.47: Non-Functional Requirement NFR-015: Data Integrity

Function Name	Data Integrity
Description	The system must ensure that data is accurate and remains unchanged except through authorized processes.
Criticality	High (Must Have)
Technical Issues	Implement thorough data validation, regular backups, and audit logging.
Cost and Schedule	Moderate
Risks	Data loss or corruption may occur if integrity safeguards fail.
Dependencies	Depends on secure data storage and validation practices.
Pre-Condition	Data has been entered into the system.
Post-Condition	Data remains consistent and reliable throughout its lifecycle.

3.7.16 Error Handling

Table 3.48: Non-Functional Requirement NFR-016: Error Handling

Function Name	Error Handling
Description	The system should detect, log, and display clear error messages for any failures, aiding in troubleshooting and repair.
Criticality	High (Must Have)
Technical Issues	Implement detailed logging and user-friendly error alerts.
Cost and Schedule	Low
Risks	Inadequate error handling could complicate troubleshooting.
Dependencies	Depends on robust software design.
Pre-Condition	An error occurs during system operation.
Post-Condition	Errors are captured in logs and communicated clearly to users.

3.7.17 Data Encryption

Table 3.49: Non-Functional Requirement NFR-017: Data Encryption

Function Name	Data Encryption
Description	Sensitive data must be encrypted during storage and transmission to protect confidentiality.
Criticality	High (Must Have)
Technical Issues	Use industry-standard encryption protocols (e.g., AES, TLS).
Cost and Schedule	Moderate
Risks	Data may be compromised if encryption is not properly implemented.
Dependencies	Depends on secure communication protocols.
Pre-Condition	The system handles sensitive user data.
Post-Condition	All sensitive data is encrypted and secure.

3.7.18 Regulatory Compliance

Table 3.50: Non-Functional Requirement NFR-018: Regulatory Compliance

Function Name	Regulatory Compliance
Description	The system must comply with all applicable legal and industry standards for data protection and privacy.
Criticality	High (Must Have)
Technical Issues	Implement policies and controls to meet regulatory requirements.
Cost and Schedule	Moderate
Risks	Non-compliance may result in legal penalties and loss of user trust.
Dependencies	None
Pre-Condition	The system processes regulated data.
Post-Condition	The system meets all regulatory standards and passes audits.

3.7.19 Continuous Monitoring

Table 3.51: Non-Functional Requirement NFR-019: Continuous Monitoring

Function Name	Continuous Monitoring
Description	The system should be monitored in real time to track performance, security, and availability.
Criticality	High (Must Have)
Technical Issues	Employ monitoring tools, set up alerts, and analyze logs to detect issues rapidly.
Cost and Schedule	Moderate
Risks	Failure to detect issues in time could lead to extended downtime.
Dependencies	Depends on monitoring software and infrastructure.
Pre-Condition	The system is live.
Post-Condition	Any issues are detected and responded to quickly.

3.7.20 Continuous Improvement

Table 3.52: Non-Functional Requirement NFR-020: Continuous Improvement

Function Name	Continuous Improvement
Description	The system should support ongoing updates and refinements based on user feedback and technological advancements.
Criticality	High (Must Have)
Technical Issues	Set up agile development processes and feedback loops.
Cost and Schedule	Moderate
Risks	Without regular updates, the system may quickly become outdated.
Dependencies	Depends on ongoing user feedback and development efforts.
Pre-Condition	The system is in active use and feedback is collected.
Post-Condition	The system is regularly updated and improved over time.

3.8 Preliminary Object-Oriented Domain Analysis

This section provides a high-level view of the system's domain model. The following class diagram illustrates the main objects in the system and their relationships.

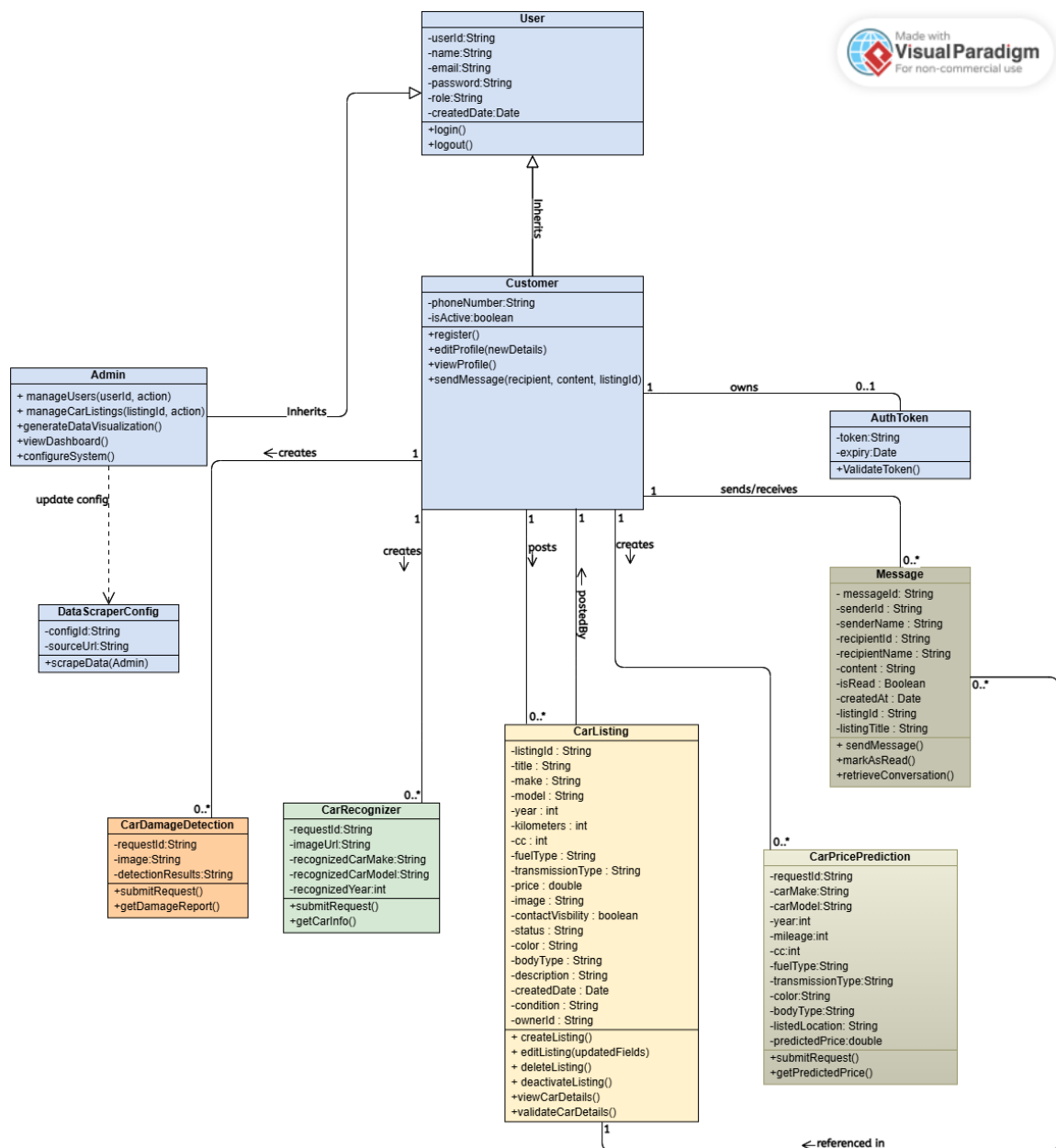


Figure 3.5: Class Diagram

3.8.1 Inheritance Relationships

This section shows the primary inheritance hierarchy for the system, illustrating where one class is a kind of another. For example, the **Admin** class inherits from the **User** class. Below is a diagram illustrating these relationships.

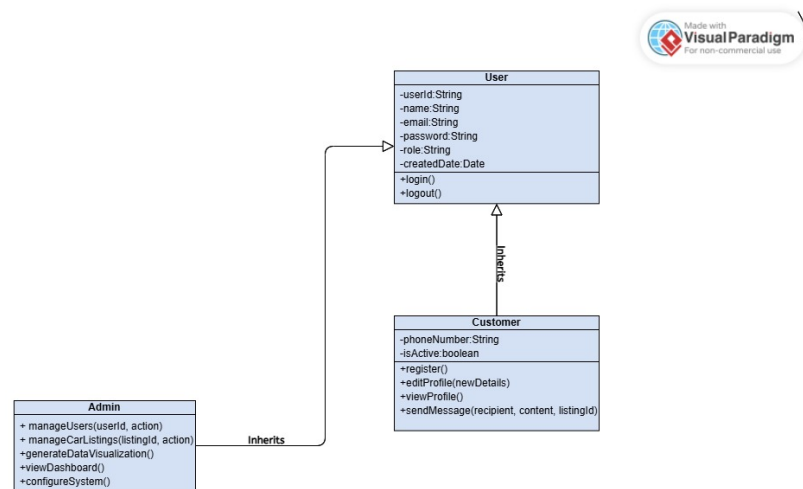


Figure 3.6: Inheritance Relations

3.8.2 Class Descriptions

Each class is described using the following structure:

- **Class Name:** Name of the class.
- **Purpose:** Role of the class in the system.
- **Attributes:** Data or properties stored in the class.
- **Methods:** Actions or operations that the class can perform.
- **Relationships:** How the class interacts with other classes.

User

- **Purpose:** Serves as the base class for all users in the system (e.g., Admin, Customer).
- **Attributes:**
 - userId : string
 - name : string
 - email : string
 - password : string
 - role : string
 - createDate : string
- **Methods:**
 - login(): Authenticates an existing user.
 - logout(): Ends the user's active session.
-

Admin (*inherits from User*)

- **Purpose:** Administer and manage the system's users, data scraping, dashboard, etc.
- **Attributes:** - (Inherits all attributes from **User**; no new fields shown here.)
- **Methods:** - **manageUsers()**: Manages user accounts (activate, deactivate, assign roles).
- **manageCarListings()**: Views/edits/deletes car listings system-wide.
- **generateDataVisualization()**: Generate charts/graphs from system data.
- **viewDashboard()**: Displays the admin dashboard.
- **configureSystem()**: Updates system configuration (scraper settings, etc.).
- **Abstract or Concrete:**
- Concrete: **Admin** can represent actual admin accounts.

AuthToken

- **Purpose:** Represents an authentication token (e.g., JWT) to manage user sessions.
- **Attributes:** - token : string
- expiry : string
- **Methods:** - **validateToken()**: Checks whether the token is valid (correct signature, not expired).
- **Abstract or Concrete:**
- Concrete: **AuthToken** objects exist to handle session state.

Message

- **Purpose:** Encapsulates messaging data sent between users.
- **Attributes:** - messageId : string
- senderId : string
- senderName : string
- recipientId : string
- recipientName : string
- content : string
- isRead : bool
- createdAt : string
- listingId : string
- listingTitle : string
- **Methods:** - **sendMessage()**: Sends or stores a new message.
- **markAsRead()**: Marks a message's status as read.

- **retrieveConversation()**: Retrieves message history with a specific user/listing.
- **Abstract or Concrete:**
- Concrete: **Message** is directly created whenever a user sends a new message.

CarListing

- **Purpose:** Stores metadata about a particular car listing.
- **Attributes:**
 - listingId : string
 - title : string
 - make : string
 - model : string
 - year : int
 - kilometers : int
 - cc : int
 - fuelType : string
 - transmissionType : string
 - color : string
 - condition : string
 - chatOption : string
 - price : float
 - description : string
 - images : string[]
 - owner_id : string
- **Methods:**
 - **createListing()**: Creates a new car listing in the system.
 - **editListing()**: Edits data in an existing listing.
 - **deleteListing()**: Deletes (or deactivates) the listing.
 - **viewCarDetails()**: Shows details of the car listing.
 - **validateCarDetails()**: Checks that required fields are filled out correctly.
- **Abstract or Concrete:**
- Concrete: **CarListing** is used for each listing.

CarPricePrediction

- **Purpose:** Handles requests for price prediction (via an ML model) for a specific car.
- **Attributes:**
 - requestId : string
 - carMake : string

- carModel : string
- year : int
- mileage : int
- cc : int
- fuelType : string
- transmissionType : string
- color : string
- bodyType : string
- listedLocation : string
- expectedPrice : float
- predictedPrice : float
- **Methods:** - submitRequest(): Submits user's input to the prediction service.
- getPredictedPrice(): Returns the final predicted price.
- **Abstract or Concrete:**
- Concrete: A CarPricePrediction object corresponds to a single user request for valuation.

CarDamageDetection

- **Purpose:** Manages the process of detecting car damage from an uploaded image.
- **Attributes:** - requestId : string
- image : string
- detectionResults : string
- **Methods:** - submitRequest(): Uploads the image and triggers damage detection.
- getDamageReport(): Retrieves summary of detected damages.
- **Abstract or Concrete:**
- Concrete: Each damage-detection request is represent as a CarDamageDetection object.

CarRecognizer

- **Purpose:** Identifies a car's make, model, and year from an uploaded image.
- **Attributes:** - requestId : string
- imageUrl : string
- recognizedCarMake : string
- recognizedCarModel : string
- recognizedYear : int

- **Methods:** - `submitRequest()`: Sends the image to the recognition service.
- `getCarInfo()`: Retrieves recognized details from the system.
- **Abstract or Concrete:**
- Concrete: `CarRecognizer` is used whenever a user uploads a car photo to identify.

DataScraperConfig

- **Purpose:** Stores admin-configurable settings for data scraping tasks.
- **Attributes:** - `configId` : string
- `sourceUrl` : string
- **Methods:** - `scrapeData()`: Initiates scraping of car data from external sources.
- **Abstract or Concrete:**
- Concrete: Each instance is a record for a scraping configuration.

3.8.2.1 List of Superclasses

- **User:** This is the base class for everyone using the system. It stores common details such as `name`, `email`, `password`, `role`, and `createDate`. It also contains basic methods like `register()`, `login()`, and `logout()`.

3.8.2.2 List of Subclasses

- **Admin:** Inherits all the user properties and behaviors from **User** and is give additional privileges.
- **Customer:** Inherits all the user properties and behaviors from **User** and is give additional privileges.

3.8.2.3 Purpose

- **User:** Serves as the foundation for managing user information and authentication.
- **Admin:** Represents a privileged user capable of managing accounts, car listings, and system configurations.
- **AuthToken:** Manages session tokens (e.g., JWT) to keep users securely logged in.
- **CarListing:** Represents a vehicle listing, storing details such as title, make, model, year, engine displacement, price, images, and more.

- **Message**: Supports communication between users about listings (e.g., inquiries between buyers and sellers).
- **CarPricePrediction**: Accepts user input (e.g., car details) and returns a price prediction using a machine learning model.
- **CarDamageDetection**: Processes an uploaded car image to identify any damage using a YOLOv8-based analysis.
- **CarRecognizer**: Analyzes a car image to automatically detect the car's make, model, and year (using, for example, a DenseNet201 model).
- **DataScraperConfig**: Contains settings for scraping external car data and is used by admins to update system data.

3.8.2.4 Collaborations

- **User** is the parent class for **Admin**; every admin is a user with all the common properties and methods.
- **Admin** uses **DataScraperConfig** to initiate external data scraping and manages **CarListing** (for editing or deleting listings) and **Message** (for monitoring communications).
- A **User** creates a **CarListing** which can later be edited or deleted by the listing owner or an **Admin**.
- **Message** objects are used for communication between users regarding a specific **CarListing**.
- The service classes **CarPricePrediction**, **CarDamageDetection**, and **CarRecognizer** process specific requests and return results (price predictions, damage reports, or recognized car details).
- **AuthToken** is associated with each **User** to manage secure sessions.

3.8.2.5 Attributes

- **User**
 - **name** (e.g., "mostafa h")
 - **email** (e.g., "mostafa@example.com")
 - **phone** (e.g., "1234567890")
 - **password** (hashed string)

- role (e.g., "admin" or "user")
 - isActive (boolean)
 - createDate (date)
- **AuthToken**
 - token (string)
 - expiry (date)
- **CarListing**
 - listingId (unique ID)
 - title (e.g., "Family SUV for Sale")
 - make (e.g., "Toyota")
 - model (e.g., "RAV4")
 - year (e.g., 2021)
 - cc (e.g., 2000)
 - fuelType (e.g., "Gasoline")
 - price (e.g., 15000)
 - image (URL or filename)
 - contactVisibility (boolean)
 - status (e.g., "active", "sold")
 - color (e.g., "White")
 - bodyType (e.g., "SUV")
 - description (text)
 - createdAt (date)
 - condition (e.g., "Used" or "New")
 - ownerId (user reference)
- **Message**
 - messageId (unique ID)
 - senderId and recipientId (user IDs)
 - content (text)
 - isRead (boolean)
 - createdAt (timestamp)
 - listingId (optional, if linked to a listing)
- **CarPricePrediction**

- requestId (unique ID)
- carMake, carModel (strings)
- year (integer)
- cc (integer)
- mileage (integer)
- fuelType (string)
- transmissionType (string)
- color (string)
- bodyType (string)
- listedLocation (string)
- expectedPrice (integer)
- predictedPrice (integer)

- **CarDamageDetection**

- requestId (unique ID)
- image (URL or filename)
- detectionResults (text)

- **CarRecognizer**

- requestId (unique ID)
- imageUrl (URL or filename)
- recognizedCarMake (string)
- recognizedCarModel (string)
- recognizedYear (integer)

- **DataScraperConfig**

- configId (unique ID)
- sourceUrl (string)

3.8.2.6 Operations

- **User**

- register(name, email, phone, password): Creates a new user account.
- login(email, password): Authenticates the user and initiates a session.

- `logout()`: Ends the current session.
 - `editProfile(newDetails)`: Updates the user's information.
- **Admin** (inherits from **User**)
 - `manageUsers(userId, action)`: For example, to block or activate user accounts.
 - `manageCarListings(listingId, action)`: To edit or delete a listing.
 - `generateDataVisualization()`: Creates charts/graphs from system data.
 - `viewDashboard()`: Displays system metrics and summaries.
 - `configureSystem()`: Adjusts settings, such as scraping configurations.
- **AuthToken**
 - `validateToken()`: Verifies whether the current token is still valid.
- **CarListing**
 - `createListing()`: Adds a new car listing.
 - `editListing(updatedFields)`: Updates listing details.
 - `deleteListing()`: Removes the listing from the marketplace.
 - `deactivateListing()`: Marks the listing as inactive without a full delete.
 - `viewCarDetails()`: Returns the listing's complete details.
 - `validateCarDetails()`: Checks that all listing details are correct.
- **Message**
 - `sendMessage(recipient, content, listingId)`: Sends a message to another user.
 - `markAsRead()`: Updates the message's status to "read."
 - `retrieveConversation(userId)`: Retrieves the conversation history with a specific user.
- **CarPricePrediction**
 - `submitRequest()`: Processes the user's input for price prediction.
 - `getPredictedPrice()`: Returns the price predicted by the machine learning model.

- **CarDamageDetection**

- `submitRequest()`: Submits an image for damage analysis.
- `getDamageReport()`: Retrieves the damage detection results.

- **CarRecognizer**

- `submitRequest()`: Uploads an image for recognition.
- `getCarInfo()`: Returns the recognized car details.

- **DataScraperConfig**

- `scrapeData(Admin)`: Initiates external data scraping using the config; requires admin privileges.

3.8.2.7 Constraints:

This section lists the constraints upon the general state or behavior of instances for each class.

- **User:**

- Each user's email must be unique.
- Passwords must meet basic security standards (e.g., at least 8 characters with a mix of letters, numbers, and symbols).
- The `role` attribute must be one of the predefined values (e.g., `"admin"`, `"user"`).

- **AuthToken:**

- The token must follow a valid JWT format.
- The token's expiry must be set to a reasonable period (for example, no longer than 30 days from issuance).

- **CarListing:**

- The listing title cannot be empty.
- The price must be a positive number.
- The car year must be within a realistic range (for instance, between 1900 and the current year).
- The `contactVisibility` flag should be a boolean value.

- **Message:**

- Message content cannot be empty.

- Both sender and recipient IDs must correspond to valid **User** instances.
- **CarPricePrediction:**
 - All required fields (e.g., carMake, carModel, year, cc, mileage) must be provided.
 - The predicted price must be a non-negative number.
- **CarDamageDetection:**
 - Only valid image formats (e.g., jpg, png) should be accepted for analysis.
- **CarRecognizer:**
 - The uploaded image must be in a supported format.
 - The recognized details (make, model, year) must correspond to known vehicle data.
- **DataScraperConfig:**
 - The source URL must be a valid URL.
 - Only an admin is allowed to modify or update the data scraper configuration.

3.9 Operational Scenarios

3.9.1 Scenario 1: User Registers an Account

- **Use Cases Covered:** Registration, Validate Registration Input
- **Actor:** User
- **Preconditions:**
 - The user is on the registration page (not logged in).
- **Steps:**
 1. User provides personal details (name, email, phone, password).
 2. System validates the input (format, uniqueness of email).
 3. If valid, system creates a new user account (registration successful).
 4. System may automatically log the user in or require login confirmation.
- **Postconditions:**

- A new account is created.
- The user can now log in with these credentials.

3.9.2 Scenario 2: User Logs In to the System

- **Use Cases Covered:** Login, Validate Credentials
- **Actor:** User
- **Preconditions:**
 - User already has an active account.
- **Steps:**
 1. User enters login credentials (email, password).
 2. System checks the credentials against stored data (validates format, compares password).
 3. If successful, user is authenticated and receives a valid session/token.
 4. User is redirected to their personal dashboard or homepage.
- **Postconditions:**
 - User is logged in and can access features requiring authentication.
 - User can manage their profile, create listings, etc.

3.9.3 Scenario 3: User Browses, Filters, and Searches Listings

- **Use Cases Covered:** Browse Car Listing, Filter Listings, Search Listings
- **Actor:** User
- **Preconditions:**
 - User is logged in (though browsing might also be allowed if your system permits guests).
- **Steps:**
 1. User navigates to the car listings page.
 2. System displays all active car listings.
 3. (Optional) User applies filters (make, model, year, price range, etc.).
 4. (Optional) User enters a search query to refine listings further.
 5. System updates the displayed listings accordingly.
- **Postconditions:**
 - User sees a curated list of cars matching their search and filter criteria.

3.9.4 Scenario 4: User Requests a Car Price Prediction

- **Use Cases Covered:** Car Price Prediction, Validate Price Prediction Input, Price Prediction Calculation
- **Actor:** User
- **Preconditions:**
 - User is logged in.
 - User has access to the price prediction feature.
- **Steps:**
 1. User navigates to the “Price Prediction” page.
 2. User enters car details (make, model, year, mileage, etc.).
 3. System validates inputs (e.g., no negative mileage).
 4. System runs the price prediction model (XGBoost or similar).
 5. The system displays a predicted price range for the car.
- **Postconditions:**
 - User is shown the estimated market value for the specified car.

3.9.5 Scenario 5: User Detects Car Damage via Image

- **Use Cases Covered:** Car Damage Detection, Car Damage Processing
- **Actor:** User
- **Preconditions:**
 - User is logged in.
 - User has a photo of the car to upload.
- **Steps:**
 1. User uploads a car image on the damage detection page.
 2. System sends image to the YOLOv5/YOLOv8-based damage detection model.
 3. The model processes the image and identifies potential damage areas.
 4. The system returns a summary (e.g., “front bumper damage detected”).
- **Postconditions:**
 - User receives an automated damage analysis for that car image.

3.9.6 Scenario 6: User Identifies Car Make/Model via Image

- **Use Cases Covered:** Car Recognizer, Car Recognition Analysis
- **Actor:** User
- **Preconditions:**
 - User is logged in.
 - User has a car photo to upload for recognition.
- **Steps:**
 1. User uploads the car photo to the “Car Recognizer” page.
 2. System processes the image using a DenseNet or similar CNN model.
 3. The model outputs recognized make, model, and possibly approximate year.
 4. The system displays the recognized details to the user.
- **Postconditions:**
 - The user sees a guess for the car’s make, model, and year based on the photo.

3.9.7 Scenario 7: User Manages a Car Listing (Create, View, Edit, Delete)

- **Use Cases Covered:** Create Car Listing, Validate Car Details, View Car Details, Edit Listing, Delete Listing, My Listed Cars
- **Actor:** User
- **Preconditions:**
 - User is logged in.
- **Steps (Create):**
 1. User navigates to “List a Car.”
 2. User fills out listing details (title, make, model, year, price, images).
 3. System validates the data (mandatory fields, data types).
 4. System saves the new listing and displays a success message.
- **Steps (View/Edit/Delete):**
 1. User goes to “My Listed Cars.”
 2. User selects a listing to view or edit its details.
 3. If editing, user updates fields; system re-validates and saves.

4. If deleting, user confirms deletion; system removes or deactivates the listing.

- **Postconditions:**

- Car listing is created, updated, or removed as requested.
- User can view or manage all their listings in “My Listed Cars.”

3.9.8 Scenario 8: User Manages Profile

- **Use Cases Covered:** Manage User Profile, Edit Profile, Validate Profile Information

- **Actor:** User

- **Preconditions:**

- User is logged in.

- **Steps:**

1. User navigates to their profile section.
2. User edits personal info (name, phone, etc.).
3. System checks that new info meets validation rules (phone must be numeric, etc.).
4. System saves the updated info.

- **Postconditions:**

- User’s profile is updated and stored in the system.
- Changes are reflected immediately on subsequent views.

3.9.9 Scenario 9: User Exchanges Messages

- **Use Cases Covered:** Messaging System, Validate Message Content

- **Actor:** User

- **Preconditions:**

- User is logged in and has an interest in a specific listing or needs to message another user.

- **Steps:**

1. User opens the messaging interface (e.g., from a listing page or user profile).

2. User composes a message content.
3. System validates the message (e.g., no forbidden words, not empty).
4. Upon sending, system stores the message and notifies the recipient.
5. The recipient can read the message and reply similarly.

- **Postconditions:**

- Messages are saved and visible in the conversation thread.
- Both sender and recipient can view or respond as needed.

3.9.10 Scenario 10: Admin Views Dashboard and Manages Car Listings

- **Use Cases Covered:** View Admin Dashboard, Display Dashboard Stats, Manage Car Listings, Deactivate Listing

- **Actor:** Admin

- **Preconditions:**

- Admin is logged in with elevated privileges.

- **Steps:**

1. Admin navigates to the “Admin Dashboard.”
2. System displays key statistics: total users, listings, recent activity.
3. Admin selects “Manage Car Listings” to see all user listings.
4. Admin can edit or deactivate (delete) any problematic or invalid listing.

- **Postconditions:**

- Admin sees updated stats in real time.
- Inappropriate listings can be removed or marked inactive.

3.9.11 Scenario 11: Admin Scrapes External Data

- **Use Cases Covered:** Manage Data Scraping, Automated Data Collection

- **Actor:** Admin

- **Preconditions:**

- Admin is logged in.
- External website scraping is configured (URLs, scraper settings).

- **Steps:**

1. Admin goes to “Manage Data Scraping” in the dashboard.
2. Admin configures or confirms the source URLs for data.
3. Admin initiates the scraping process.
4. The system scrapes data from external sites, standardizes it, and stores it locally.

- **Postconditions:**

- New or updated car listing data from external sources is integrated into the system.

3.9.12 Scenario 12: Admin Views Data Visualization

- **Use Cases Covered:** Manage Data Visualization, Generate Data Visualization

- **Actor:** Admin

- **Preconditions:**

- Admin is logged in.
- There is sufficient data in the system (listings, user interactions).

- **Steps:**

1. Admin selects the “Data Visualization” tab or section in the dashboard.
2. System fetches relevant stats (car makes, models, yearly trends, fuel distribution, etc.).
3. System generates interactive graphs or charts (using Recharts or another library).
4. Admin can filter or drill down on any specific data set.

- **Postconditions:**

- Admin is presented with visual insights about the marketplace, user activity, or scraped data.
- The admin can make data-driven decisions (like focusing on certain popular makes or analyzing listing trends).

3.9.13 Use Case Diagram

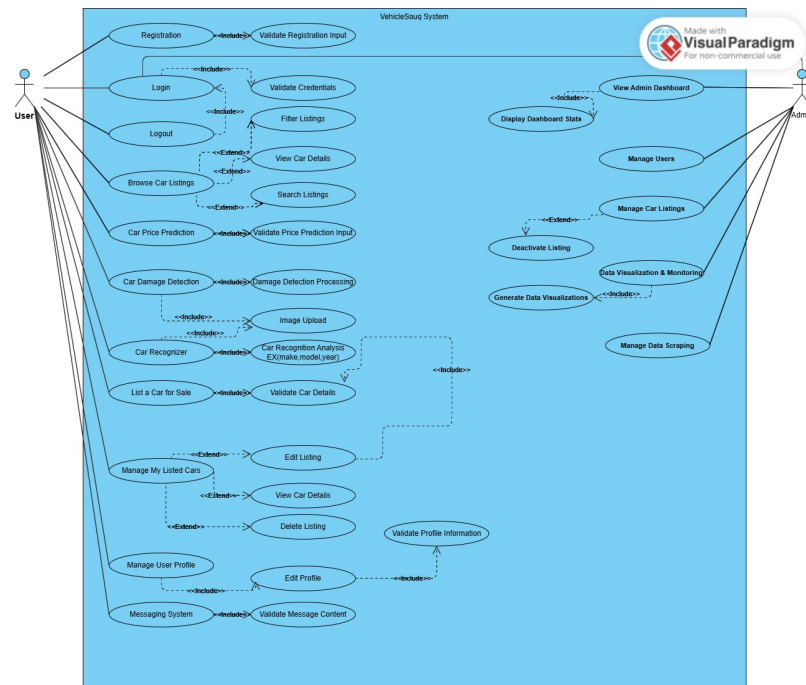


Figure 3.7: Use Case Diagram for System Operations

3.9.14 Use Case Diagram

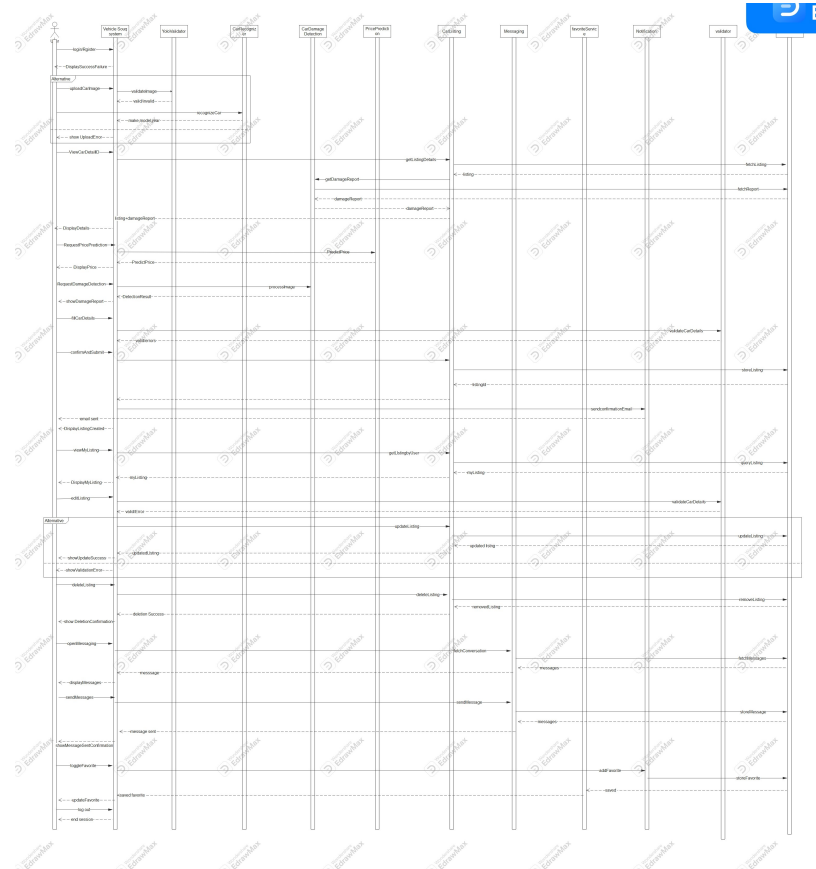


Figure 3.8: Sequence Diagram User

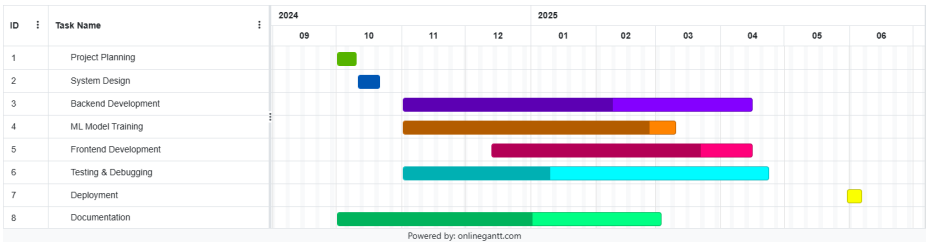


Figure 3.10: Gantt Chart for Project Plan

3.11 Preliminary Budget Adjusted

Total initial budget: 20,000 EGP

3.12 Appendices

3.12.1 Definitions, Acronyms, Abbreviations

- **SRS:** Software Requirements Specification. A document that describes the software system that will be developed.
- **ML:** Machine Learning. A data analysis technique that builds analytical models automatically.
- **API:** Application Programming Interface. A set of rules that allows different software programs to talk to each other.
- **GUI:** Graphical User Interface. The visual part of a software application that allows users to interact with it using images and icons.

Chapter 4

Design

4.1 Design Guidelines

The Software Design Document (SDD) for a complete vehicle listing and analysis platform is presented in this document. The architecture combines machine learning features like price prediction, damage detection, and car model recognition with conventional web components.

This SDD is classified as:

1. **Application Oriented:** Application of a Selected Approach

This document describes how the system was created to satisfy technical objectives as well as user needs. The platform's flexible, scalable architecture allows users to list cars, chat, identify car models, detect damage, and predict prices.

Through iterative development and feedback, the design changed, making important decisions like using MongoDB for flexible data handling and FastAPI for speed.

In addition to explaining how the system functions, this chapter highlights what was learned, difficulties faced, and choices made to guarantee performance, usability, and maintainability.

4.2 Introduction

4.2.1 Purpose

The system architecture and design of an AI car listing platform are described in this Software Design Document (SDD). It is meant to help developers, managers, and maintainers comprehend the system's operation and the rationale behind important design decisions.

4.2.2 Scope

This platform offers advanced features like the following in addition to allowing users to register, log in, and post used car listings:

- using a machine learning model that has been trained to forecast the car's market value.
- using computer vision to identify physical damage in uploaded photos.
- Recognizing the make, model, and year of a car from an image.
- allowing direct chat between users on the platform.
- enabling administrators to keep an eye on system activity and extract information from outside sources.

The system is constructed using a MongoDB database, a FastAPI backend, and a React frontend. It is intended to be scalable, modular, and simple to expand with future smarter features.

4.2.3 Overview

This paper outlines the architecture of the system, as well as various important modules such as listings, messaging, authentication, and machine learning service. Everything in the complex three-tier architecture diagram was created, tested, and optimized for usability and performance.

4.2.4 Reference Material

No specific reference materials.

Term	Definition
Software Design Document (SDD)	used as the main document that explains the structure and interactions of a software system.
Design Entity	A component of the system that plays a specific role and interacts with other components, such as a module, function, or service.
SPA	A single-page application is a web application that loads just one HTML page and updates its content dynamically without requiring a reload.
JWT	A secure method of sending authentication information between the client and server is the JSON Web Token.
YOLOv8	An object detection model that detects damage in uploaded car photos in real time.
FastAPI	A cutting-edge web framework for creating scalable and quick APIs that is based on Python.
MongoDB	a flexible, document-oriented NoSQL database for storing data.
XGBoost	Car price prediction using an optimized gradient boosting machine learning algorithm.

Table 4.1: Definitions and Acronyms

4.2.5 Definitions and Acronyms

4.3 System Overview

This project is a AI online platform that makes it easier and more confident for people to buy and sell used cars. It has intelligent features driven by machine learning models in addition to standard listing and browsing capabilities:

- utilizing XGBoost to forecast vehicle prices based on user input.
- identifying obvious damage in uploaded photos (using YOLOv8).
- using DenseNet201 to automatically identify the year, make, and model of an automobile from a picture.
- enabling users to discuss a listing in a platform chat.
- providing administrators the ability to import listings from outside sources and keep an eye on activity.

The tech stack used to build the system consists of MongoDB for the database, FastAPI for the backend, and React for the frontend. To guarantee a seamless and dependable user experience, the design places a strong emphasis on responsiveness and modularity.

4.4 System Architecture

4.4.1 Architectural Design

The system follows a modular, layered architecture to support scalability, maintainability, and integration of machine learning services. It is composed of three main layers:

- **Frontend (React SPA):** The user interface where users can register, log in, browse listings, view car details, and access ML-powered features such as price prediction and damage detection.
- **Backend (FastAPI):** Handles business logic, user authentication (JWT), data processing, messaging, and communicates with machine learning models. It exposes RESTful APIs consumed by the frontend.
- **Database (MongoDB):** Stores user accounts, car listings, chat messages, and image metadata using a flexible schema.

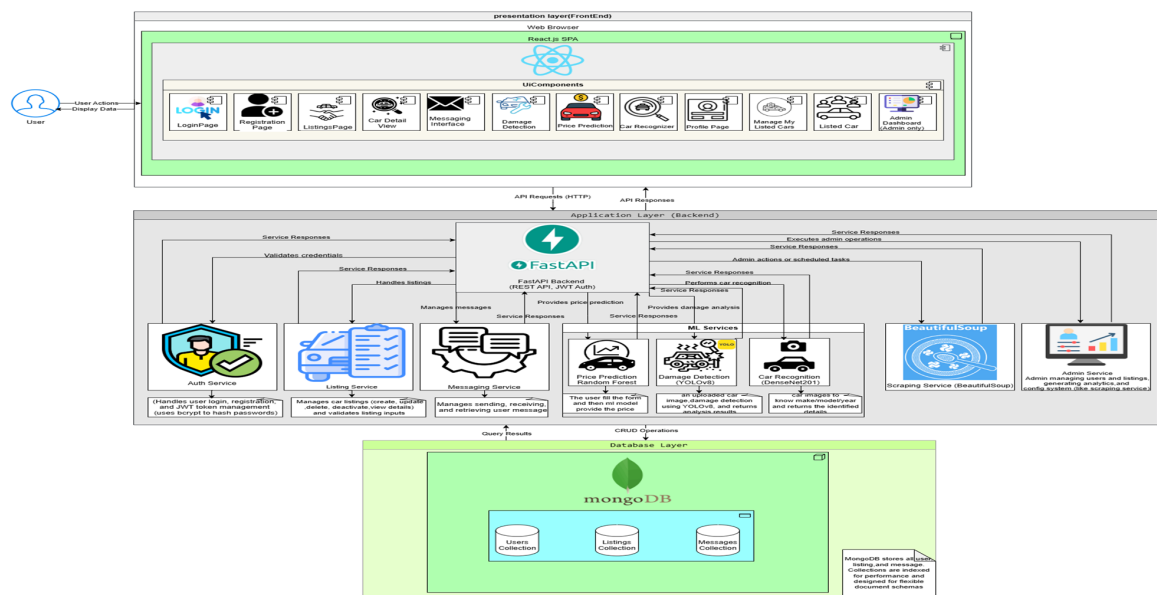


Figure 4.1: Architectural Design

4.4.2 Decomposition Description

We provide a functional breakdown of the system into three main subsystems in this subsection:

- **Web-Scraping Subsystem:** In charge of crawling target classifieds websites, sending URLs to scrapers, cleaning raw HTML, addressing fields that are missing or malformed, extracting the desired attributes (make, model, year, mileage, etc.), and exporting a combined CSV file for use as a result.
- **Price-Prediction Subsystem:** generates a final market-price estimate by applying feature-engineering and preprocessing to tabular vehicle data (from the Web-Scraping CSV or manual input), training and choosing between Random Forest and XGBoost regressors.
- **Image-Detection Subsystem:** A make-model-year prediction is produced for integration into the pricing pipeline after user-supplied car images (or batch images) are ingested, data-augmentation is applied, and refined DenseNet201 and ResNet152 classifiers are trained on the Stanford Cars dataset.

4.4.2.0.1 Top-Level Data-Flow Figure 4.2 shows the DFD for the Web-Scraping Subsystem. Figure 4.3 shows the structural decomposition of the Price-Prediction Subsystem, and Figure 4.4 shows the structural decomposition of the Image-Detection Subsystem.

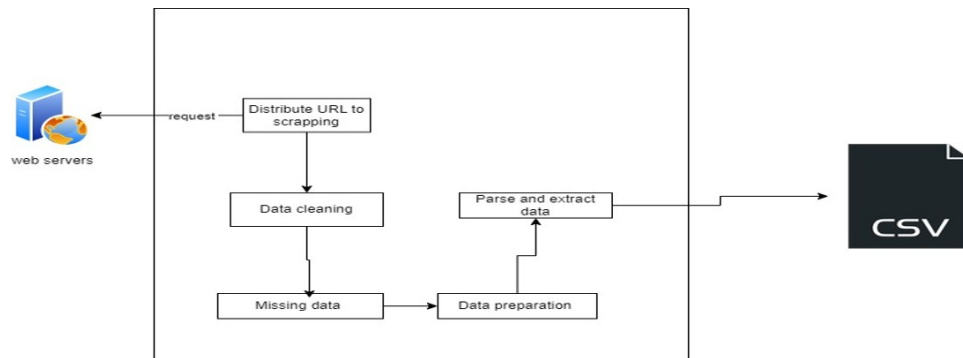


Figure 4.2: Web-Scraping System Architecture (Top-Level DFD)

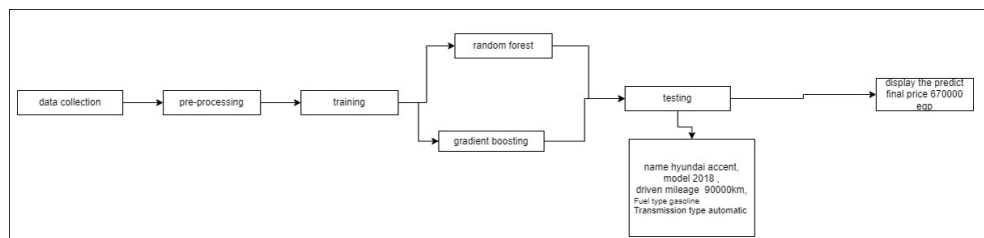


Figure 4.3: Price-Prediction Model Architecture

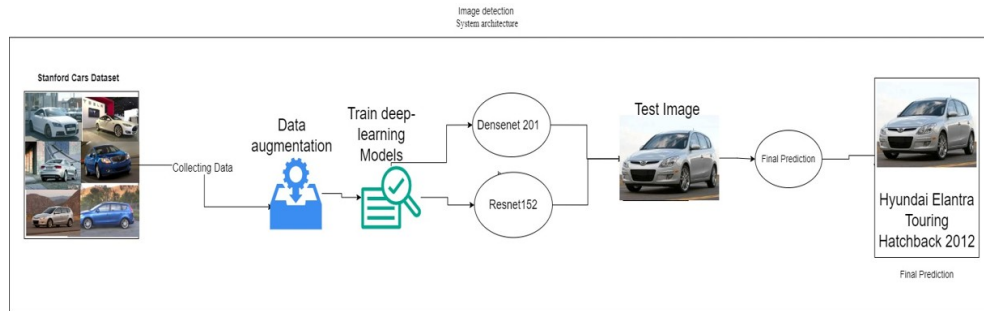


Figure 4.4: Image-Detection Model Architecture

4.4.2.0.2 Discussion

- The *Web-Scraping* URL distribution, HTML cleaning, attribute parsing, missing-data imputation, and CSV export are all included in DFD.
- The *Price-Prediction* structure diagram shows two parallel regression training branches (Random Forest, Gradient Boosting/XGBoost). During testing, these branches come together, and a price display marks the conclusion.
- Data-augmentation feeding two CNN training branches (DenseNet201, ResNet152) is depicted in the *Image-Detection* structure diagram. The outputs of these branches are combined to create the final model classification.

The data flows and processing steps that convert unprocessed web data and images into useful pricing and classification outcomes are made clear by these functional decomposition diagrams taken together. For this project, the functional DFDs and structural decomposition diagrams are sufficient to show the architecture's modularity and data pathways. However, if an object-oriented perspective is desired, each block could be further broken down into classes (e.g. `ScraperManager`, `Cleaner`, `PriceRegressor`, `CNNTrainer`, etc.).

4.4.3 Design Rationale

Scalability, modularity, and integration with machine learning services were all taken into consideration when choosing the architecture. Because of its high-performance, lightweight asynchronous API handling, FastAPI was chosen. Fast, responsive user interfaces without full-page reloads were made possible by the frontend's use of React. MongoDB was selected because of its adaptable document model, which works well with user-generated content and dynamic car listings.

Key decisions and trade-offs:

- **FastAPI over Django/Flask:** FastAPI automatically creates documentation and supports async calls. Django was heavier but had more features.
- **MongoDB over SQL:** NoSQL eliminated the need for rigid schemas or transfers and made it easier to store images, nested ML outputs, and a variety of listing formats.
- **Decoupled ML models:** ML models can be scaled or updated independently of the backend logic when they are run as distinct services.
- **REST over GraphQL:** REST was easier to test and deploy, especially for a time-constrained team.

Alternatives such as traditional SQL databases or monolithic Django applications were taken into consideration, but they introduced rigidity or complexity that was inappropriate for a system that was feature-rich, image-heavy, and ML-enhanced.

4.5 Data Design

4.5.1 Data Description

All of the platform's essential entities, including users, listings, messages, and machine learning outcomes, are stored in MongoDB. Because MongoDB can store nested JSON-like documents, it makes handling chat histories, prediction outputs, and car photos easier. Every listing includes both ML-generated metadata (such as damage areas) and structured data (such as year and price), along with a reference to the user who created it.

Key data flows:

- A listing submitted by the user is saved in 'listings' with reference to 'users'.
- Image sent to ML service → output embedded in 'listings' or saved under 'ml_results'.
- These messages are arranged by sender and recipient and are kept in 'messages' as pairs.

Entity	Field	Description
User	<code>_id</code> <code>email</code> <code>passwordHash</code>	Unique identifier User email (used for login) Hashed password for authentication
Listing	<code>_id</code> <code>userId</code> <code>make</code> <code>model</code> <code>year</code> <code>images</code>	Listing identifier Reference to listing creator Car brand (e.g., Toyota) Car model (e.g., Corolla) Year of manufacture Array of image URLs
ML Results	<code>listingId</code> <code>price_prediction</code> <code>damage_areas</code> <code>model_classification</code>	Reference to related listing Output from XGBoost model YOLOv8 bounding box results DenseNet201 output (make, model, year)
Message	<code>from</code> <code>to</code> <code>content</code> <code>timestamp</code>	Sender's user ID Receiver's user ID Message text Message send time

Table 4.2: Data Dictionary of Core System Entities

4.5.2 Data Dictionary

4.6 Component Design

4.6.1 User Registration and Authentication

Purpose: Allow users to create accounts, log in securely, and maintain session state.

Pseudo-code:

```
Function RegisterUser(name, email, phone, password):
```

```
    If email already exists:
        Return "Email in use"
    passwordHash = hash(password)
    Store user(name, email, phone, passwordHash)
    Return "Registration successful"
```

```
Function LoginUser(email, password):
```

```
    user = FindUserByEmail(email)
    If user not found or !check(password, user.passwordHash):
        Return "Invalid credentials"
    token = GenerateJWT(user.id)
    Return token
```

4.6.2 Car Recognition via Image

Purpose: Automatically detect make, model, and year from a user-uploaded car image.

Pseudo-code:

```
Function RecognizeCarImage(image):  
    image = Preprocess(image)  
    prediction = DenseNet201(image)  
    Return prediction.make, prediction.model, prediction.year
```

4.6.3 Price Prediction

Purpose: Predict car price based on either user input or recognized data.

Pseudo-code:

```
Function PredictPrice(carData):  
    features = TransformToFeatures(carData)  
    price = XGBoostModel.predict(features)  
    Return price
```

4.6.4 Car Listing Management

Purpose: Let users create, edit, and delete their car listings.

Pseudo-code:

```
Function CreateListing(userID, listingData, images):  
    Validate listingData  
    Save listing to database  
    Upload images to storage  
    Return "Listing created"
```

```
Function EditListing(userID, listingID, newData):  
    listing = GetListingByID(listingID)  
    If listing.ownerID != userID:  
        Return "Unauthorized"  
    Update listing with newData  
    Return "Listing updated"
```

```
Function DeleteListing(userID, listingID):  
    listing = GetListingByID(listingID)  
    If listing.ownerID != userID:  
        Return "Unauthorized"  
    Mark listing as deleted  
    Return "Listing deleted"
```

4.6.5 Messaging System

Purpose: Facilitate communication between buyers and sellers.

Pseudo-code:

```
Function SendMessage(senderID, receiverID, messageText):  
    message = {senderID, receiverID, messageText, timestamp}  
    Save message to database  
    Notify receiver (if online)  
    Return "Message sent"
```

```
Function GetConversation(user1, user2):  
    messages = QueryMessagesBetween(user1, user2)  
    Sort messages by timestamp  
    Return messages
```

4.6.6 Admin Dashboard Tools

Purpose: Provide system stats and management tools to admins.

Components:

- User and listing counters
- Graphs of trends (Recharts)
- Toggle user status (active/inactive)
- Listing moderation (edit/delete)

4.6.7 Data Scraping and Scheduling

Purpose: Regularly scrape online car marketplaces and update system data.

Pseudo-code:

```
Function RunScraper():  
    For each source in SourceList:  
        data = FetchHTML(source.url)  
        listings = ParseListings(data)  
        For each listing:  
            NormalizeData(listing)  
            SaveToDatabase(listing)  
    Return "Scraping complete"
```

4.7 Human Interface Design

4.7.1 Overview of User Interface

The user interface is designed to be simple and intuitive. Users can register, log in, create car listings, browse listings, and view detailed information about each car. They can also upload images to get price predictions, detect damage, and recognize car models. The system provides clear feedback through messages, predictions, and visual highlights, making it easy for users to understand and interact with all features.

4.7.2 Screen Images

The following screenshots illustrate the main user interface components of the system:

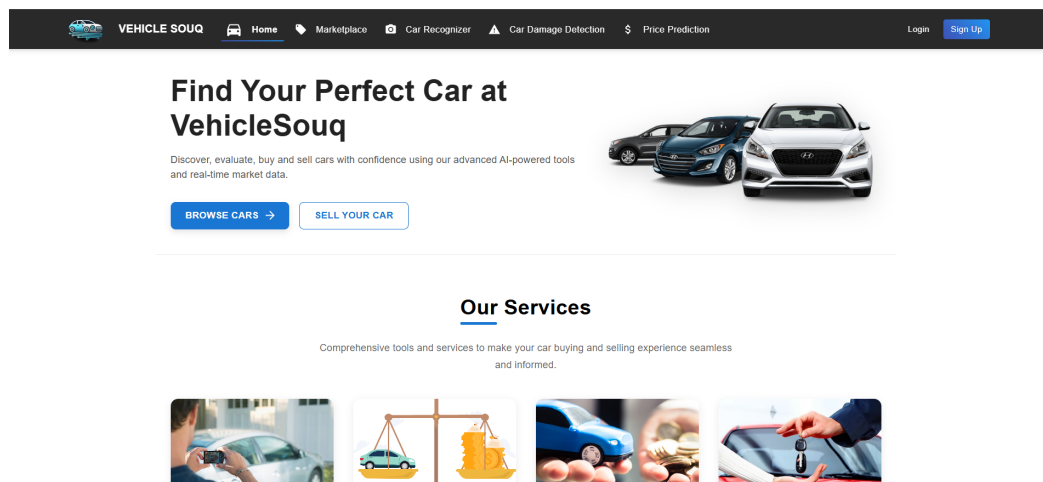


Figure 4.5: Home Page

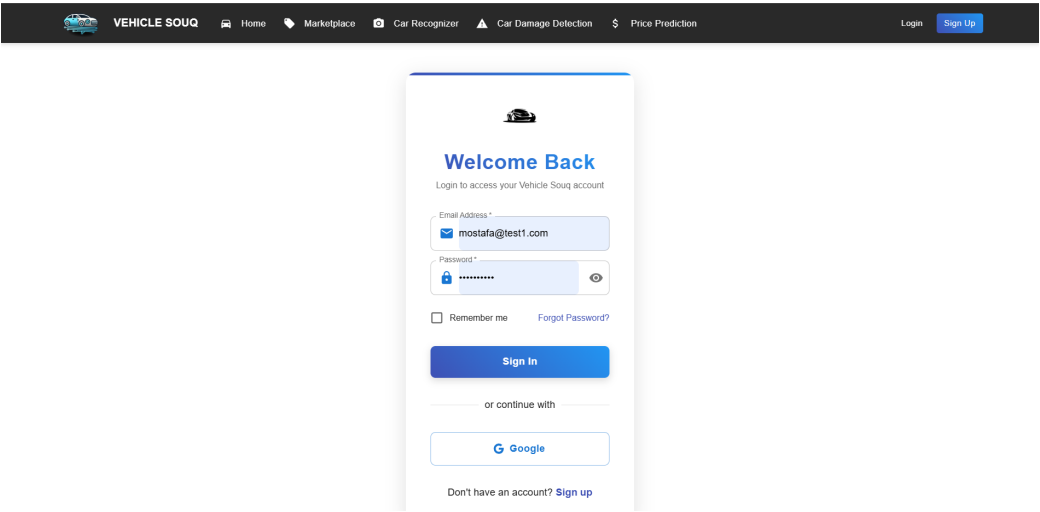


Figure 4.6: Registration Page

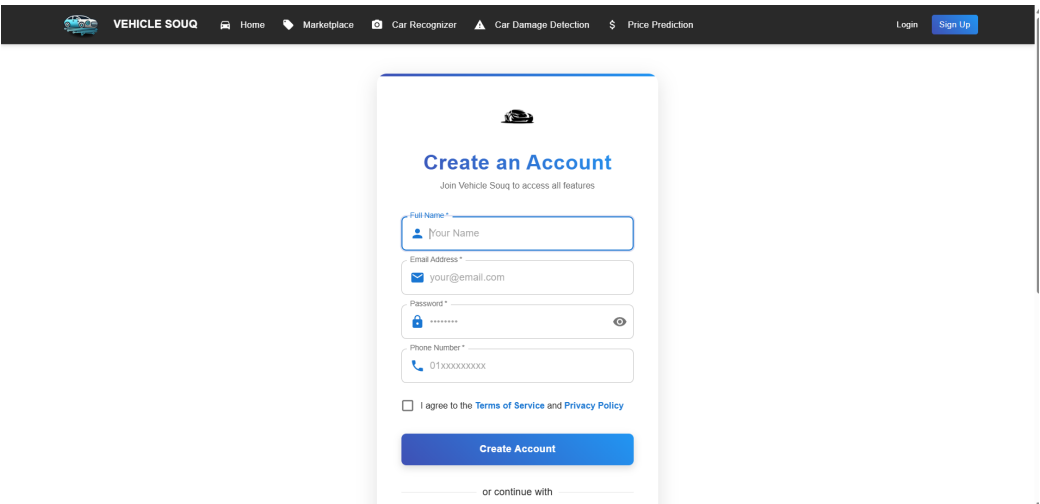


Figure 4.7: Login In

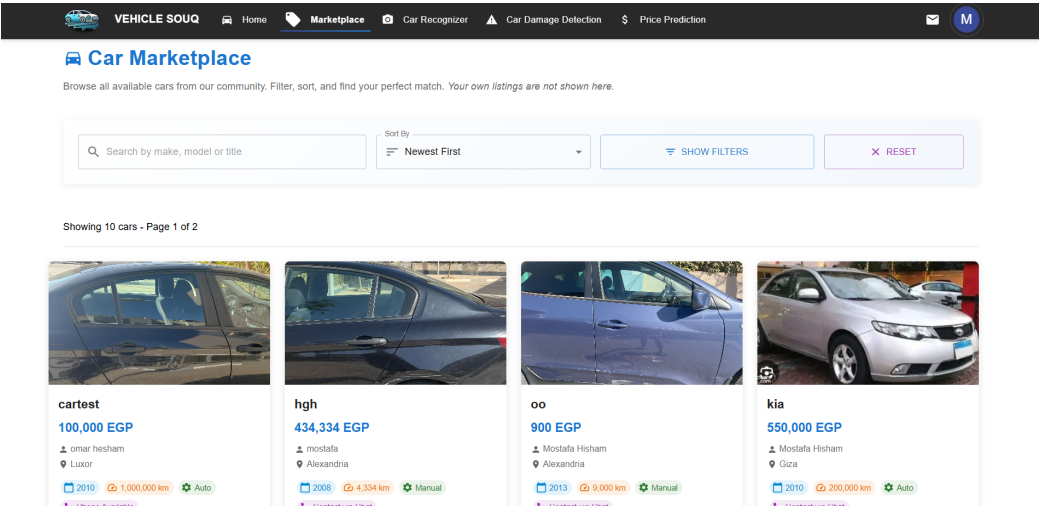


Figure 4.8: SignUp

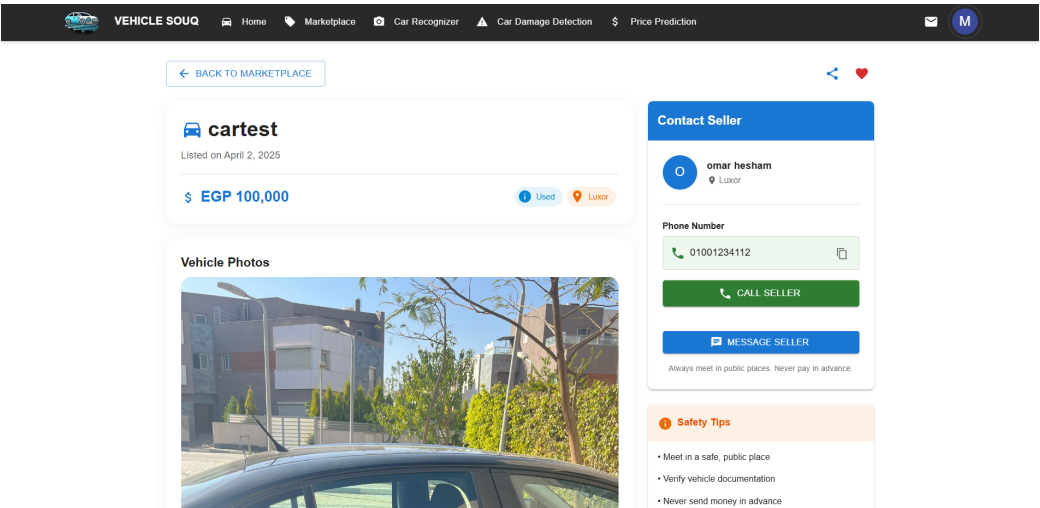


Figure 4.9: View Listing Details

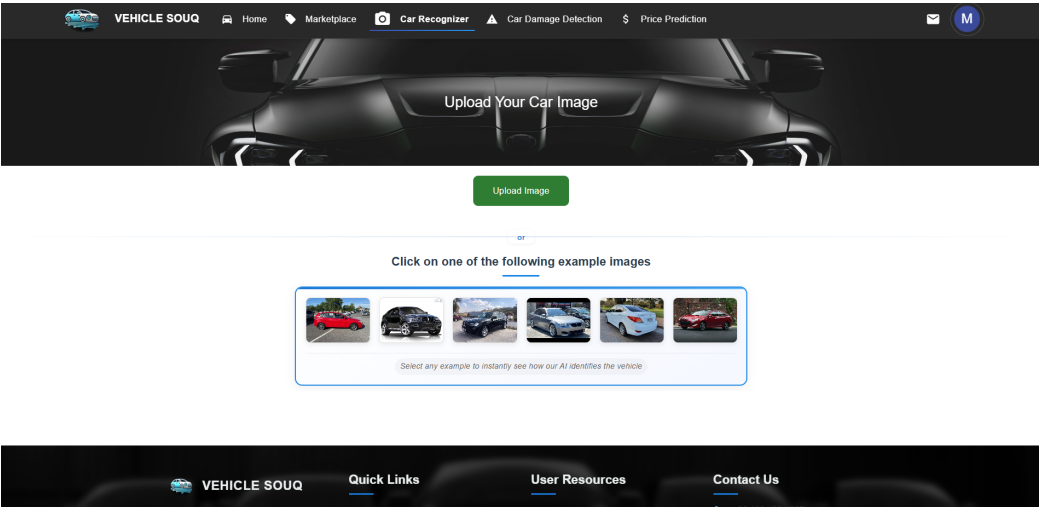


Figure 4.10: Image recognition

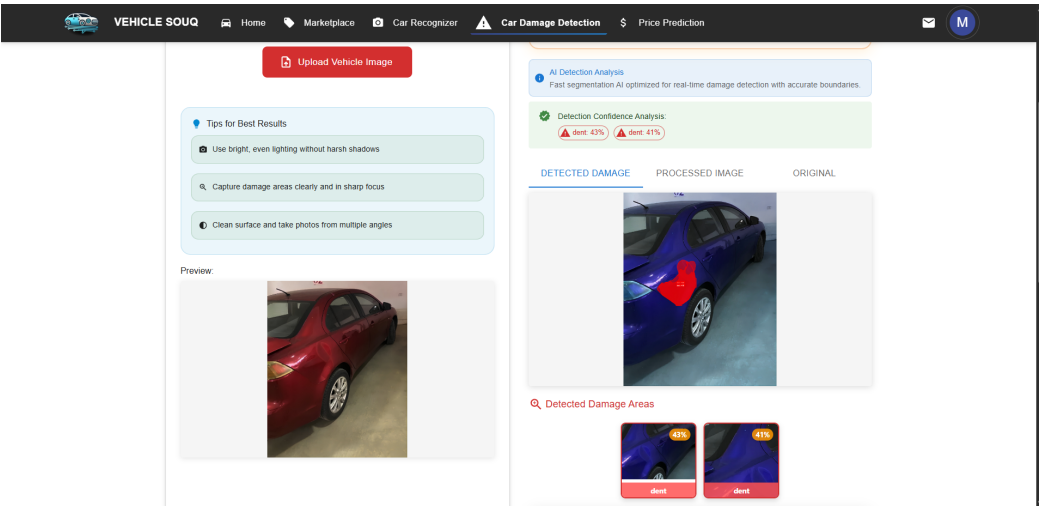


Figure 4.11: Car damage

The screenshot shows a web form titled "Enter Vehicle Details" on the left and a "Ready for Price Prediction" panel on the right. The form includes fields for Make*, Model*, Year* (set to 2025), Body Type, Kilometers* (0 km) with a slider from 0 to 300k and an "ENTER MANUALLY" button, Color, Fuel Type*, Transmission Type*, CC, Location, and Listed By. A blue "PREDICT PRICE" button is at the bottom. The right panel features a line graph icon, the text "Ready for Price Prediction", and instructions to fill the form and press "Predict Price". It also has two buttons: "Market-based pricing algorithm" and "Uses real sales data".

Figure 4.12: Price Prediction

The screenshot displays a web application interface with a dark header bar containing the "VEHICLE SOUQ" logo and navigation links: Home, Marketplace, Car Recognizer, Car Damage Detection, and Price Prediction. On the left is a "Messages" sidebar with a list of conversations. The main area shows a chat window with "Mostafa Hisham". The chat history includes messages from Mostafa Hisham and omar hesham. A "New Message" notification from Google Chrome is visible in the bottom right corner.

Figure 4.13: Messages Interface

VEHICLE SOUQ

HomeMarketplaceCar RecognizerCar Damage DetectionPrice Prediction

List Your Car

123

Vehicle DetailsSpecificationsPhotos & Price

Basic Information

Title *

Make *Model *Year *Condition *

Contact Options

Show my phone number in the listing (Not available)

Buyers will only be able to contact you through the messaging system.

Figure 4.14: User ListCar Page

VEHICLE SOUQ

HomeMarketplaceCar RecognizerCar Damage DetectionPrice Prediction

My Listings

Chevrolet Optra 2012

EGP 44,444

44,434 kmGoldLuxorMarch 29, 2025

VIEW DETAILSVIEW DETAILSVIEW DETAILS

Alfa Romeo Giulietta 2022

EGP 260

10,000 kmRedGizaMarch 31, 2025

VIEW DETAILSVIEW DETAILSVIEW DETAILS

Cadillac Escalade 2008

EGP 323,232,323

2,121,212,121 kmRedGizaMarch 31, 2025

VIEW DETAILSVIEW DETAILSVIEW DETAILS

Figure 4.15: User Listing

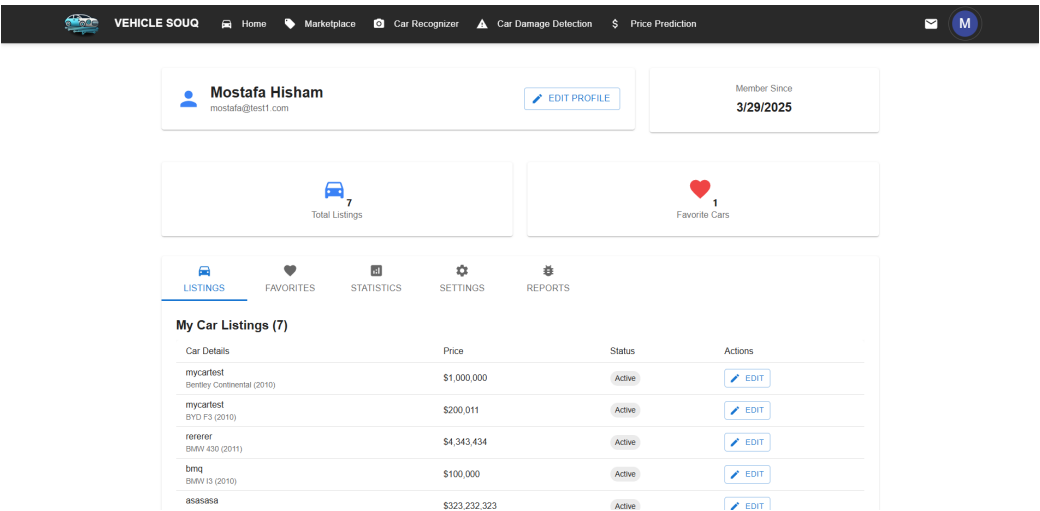


Figure 4.16: User Profile

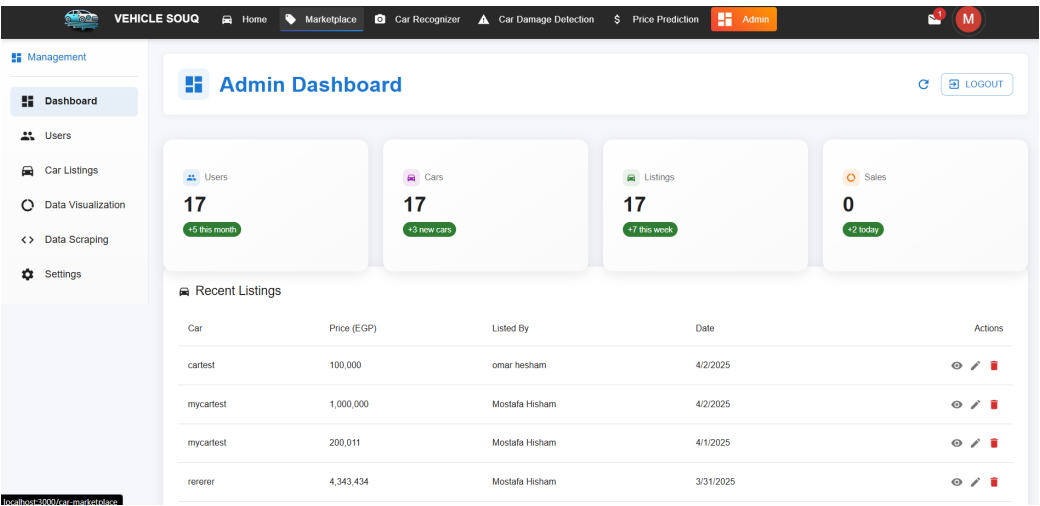


Figure 4.17: Admin Dashboard

4.7.3 Screen Objects and Actions

Several critical interface components and related user actions are included in the interface to facilitate seamless platform navigation and interaction:

- **Navigation Bar** – Login, register, dashboard, listings, and logout are all easily accessible.
- **Footer** – includes copyright, contact details, and platform details.

- **Homepage** – shows direct links to prediction and detection tools, search filters, and featured car listings.
- **Listing Cards** – Display a summary of the car’s make, model, and price; clicking brings up a more in-depth view with pictures and forecasts.
- **Upload Image Button** – enables users to upload pictures of their cars for damage detection or model recognition.
- **Damage Detection Page** – uses the bounding boxes that are returned from the machine learning model to visually highlight any damage found in an automobile image.
- **Price Prediction Form** – To get an estimated price, users enter information about their car, such as the brand, year, mileage, etc.
- **Profile Page** – shows user data, messages received, and listings that have been posted.
- **Chat Interface** – allows for real-time updates and one-on-one messaging between users.
- **Admin Panel** – Web scraping tools, content control, and user management are only accessible by administrators.

4.8 Requirements Matrix

The functional requirements (as stated in the SRS) and the system components that support or implement them are cross-referenced in this section.

Table 4.3: Requirements Traceability Matrix

SRS ID	Requirement	Component(s)
FR-001	User Registration	Auth Service, User DB
FR-002	User Login	Auth Service, JWT Handler
FR-003	User Logout	Auth Service, Token Manager
FR-004	User Profile Management	User Service, Profile DB
FR-005	Password Recovery	Auth Service, Email Service
FR-006	Car Listing Creation	Listing Service, Listings DB
FR-007	Edit Car Listing	Listing Service, Listings DB
FR-008	Delete Car Listing	Listing Service, Listings DB
FR-009	View Car Listings	Frontend, Listing Service
FR-010	Search/Filter Listings	Search Engine, UI Filter Component
FR-011	View Listing Details	Listing Service, Detail UI
FR-012	Car Price Prediction	ML Service, XGBoost Model
FR-013	Car Damage Detection	ML Service, YOLOv8 Model
FR-014	Car Recognizer	ML Service, DenseNet Model
FR-015	Car Image Validation	Image Validator, ML Service
FR-016	Send Message	Messaging Service, WebSocket/HTTP
FR-017	Retrieve Conversation History	Messaging Service, Messages DB
FR-018	Mark Message as Read	Messaging Service, Message Flags
FR-019	View System Statistics	Admin Dashboard, Metrics Aggregator
FR-020	Manage Users	Admin Dashboard, User Admin Service
FR-021	Manage Car Listings	Admin Dashboard, Listing Admin Service
FR-022	Data Visualization	Admin Dashboard, Recharts Library
FR-023	Highest Car Make Analysis	Analytics Engine, Listing DB
FR-024	Highest Car Model Analysis	Analytics Engine, Listing DB
FR-025	Fuel Type Distribution	Analytics Engine, Listing DB
FR-026	Market Trend Analysis	Analytics Engine, Trend Visualizer
FR-027	Data Scraping and Scheduling	Scraper Module, Scheduler
FR-028	Error Handling and Logging	Logging Service, Exception Handler
FR-029	Data Validation for Car Details	Validator Module, UI Form Validator
FR-030	Responsive UI/UX Design	Frontend (React), Material UI
FR-031	Account Deactivation	User Service, User Status Flag
FR-032	Car Listing Image Management	Image Upload Module, Cloud Storage

Chapter 5

Implementation

5.1 General Guidelines

We describe how we translated our design into functional code in this chapter. We concentrate on the most crucial components: the machine learning components in the backend, the APIs that drive the frontend, and the difficulties we encountered during development. We only include the most important parts of the code, not every line.

5.2 Backend Endpoints

5.2.1 1. Price Prediction

Users send car details to this endpoint and get back an estimated price:

```
@router.post("/predict_price")
async def predict_price(car_data: CarData):
    try:
        predicted_price = predict_car_price(car_data.dict())
        return {"predicted_price": predicted_price}
    except Exception as e:
        logging.error(f"Price_prediction_error:{e}")
        raise HTTPException(status_code=400, detail=str(e))

def predict_car_price(car_data):
```

```

# Turn input into a DataFrame
car_df = pd.DataFrame([car_data])
# Encode categories (make, model, etc.)
for col, le in label_encoders.items():
    car_df[col] = car_df[col].apply(
        lambda x: le.transform([x])[0] if x in le.classes_ else -1
    )
# Scale features and predict
car_scaled = scaler.transform(car_df)
return float(best_model.predict(car_scaled)[0])

```

Why this matters: Our app’s endpoint, which provides users with the fair market price, is its central component. Here, our XGBoost model achieved a **R² of 0.97** on test data.

5.2.2 2. Car Recognition

When a user uploads a photo, the make, model, and year are returned:

```

@router.post("/predict", response_model=PredictionResponse)
async def predict(file: UploadFile = File(...)):
    try:
        img = PILImage.create(io.BytesIO(await file.read()))
        pred, idx, probs = learn.predict(img)
        return PredictionResponse(
            prediction=str(pred),
            probability=float(probs[idx])
        )
    except Exception as e:
        logging.error(f"Recognition_error: {e}")
        raise HTTPException(status_code=400, detail=str(e))

```

We confirm that there is a car in the uploaded image before performing additional analyses:

Model accuracy: The accuracy of our DenseNet201 on Stanford Cars was **94.41 %**.

5.2.3 3. Car Detection Check

We confirm that there is a car in the uploaded image before performing additional analyses:

```
@router.post("/check_car", response_model=YoloResponse)
async def check_car(file: UploadFile = File(...)):
    try:
        if model is None:
            raise HTTPException(500, "YOLO_model_not_loaded")
        img = Image.open(io.BytesIO(await file.read()))
        results = model(img)
        # Look for class_id==2 (car) with confidence > 0.5
        car_detected = any(
            int(det[5])==2 and float(det[4])>0.5
            for det in getattr(results[0], "boxes", results.xyxy)[0].data
        )
        return YoloResponse(car_detected=car_detected)
    except Exception as e:
        logging.error(f"YOLO_error:{e}")
        raise HTTPException(status_code=400, detail=str(e))
```

5.2.4 4. Damage Report Retrieval

Admins and users can fetch a saved damage report:

```
@router.get("/report/{report_id}")
async def get_damage_report(report_id: str, user = Depends(get_current_user)):
    try:
        user_id = str(user["_id"])
        report = await db.damage_reports.find_one({
            "report_id": report_id,
            "user_id": user_id
        })
        if not report:
```

```

        raise HTTPException(404, "Report_not_found")
    report["_id"] = str(report["_id"])
    report.setdefault("image_results", [])
    return {"status": "success", "report": report}
except Exception as e:
    logging.error(f"Report_fetch_error: {e}")
    raise HTTPException(500, detail=str(e))

```

5.2.5 5. Car Details API

Fetch a single car's details and seller info:

```

@app.get("/api/cars/{car_id}")
async def get_car_by_id(car_id: str):
    try:
        if not ObjectId.is_valid(car_id):
            raise HTTPException(400, "Invalid_car_ID")
        car = await db.listings.find_one({"_id": ObjectId(car_id)})
        if not car:
            raise HTTPException(404, "Car_not_found")
        car["_id"] = str(car["_id"])
        # Attach seller info
        user = await db.users.find_one({"_id": ObjectId(car["user_id"])})
        if user:
            car["seller"] = {
                "username": user.get("username", "Unknown"),
                "email": user.get("email")
            }
        return car
    except HTTPException:
        raise
    except Exception as e:
        logging.error(f"Fetch_car_error: {e}")
        raise HTTPException(500, detail=str(e))

```

5.3 Machine Learning Training

We trained our image model with FastAI using data augmentation and mixup:

```
learn = cnn_learner(
    dls, densenet201,
    metrics=[accuracy, F1Score(average='weighted')],
    pretrained=True,
    cbs=[MixUp(alpha=0.4)]
)
learn.fine_tune(
    65,
    base_lr=0.0019,
    cbs=[SaveModelCallback(every_epoch=True)]
)
learn.export('final_model.pkl')
```

Key results:

- **Accuracy:** 94.41 %
- **F1 Score:** 0.9439

5.4 Common Challenges and Solutions

- **Model compatibility:** Instead of ONNX, which failed, 'export()' was needed to export FastAI models to serve in FastAPI.
- **Slow ML calls:** Latency was increased by predicting on each request. For repeated inputs during testing, we included a basic cache.
- **Bad images:** Accuracy is impacted by poor upload quality. During training, we increased augmentation (random erasing, lighting changes).
- **Large DB queries:** There were a few slow listing calls. Results were paginated, and only certain fields were returned.

5.5 Deliverables

By the end of implementation we had:

- a quick backend with APIs for image analysis and price prediction.
- a price-based trained XGBoost model ($R^2 = 0.97$).
- DenseNet201 algorithm (94 % accuracy) for models and makes.
- damage detection based on YOLOv8.
- Features of the admin dashboard, listing CRUD, and user accounts.
- A simple React frontend consuming these APIs.

5.6 Summary

The design and integration of the core code were demonstrated in this chapter. We discussed the main challenges, the training methods, the important endpoints, and how we overcame them. Now that these components are in place, the system can provide actual users with fast and reliable cars ideas.

Chapter 6

Results and Evaluation

6.1 General Rules

This section provides an overview of how well the system achieved its goals, how we verified its operation, and how certain we were of the results. We justify our test strategy, give an overview of the outcomes of all significant tests, and recommend other tests that are necessary for higher confidence. We also critically assess the methodological and technological decisions.

6.2 Test Cases Overview

We executed a total of 131 test cases across UI, API and model evaluation:

- **UI tests (Katalon):** 97 cases, 6 failures.
- **API tests (Postman):** 22 cases, 3 failures.
- **Performance testing (Built-in + Locust):** 5 cases, 0 failures.
- **Overall:** 124 executed, 9 failed.

6.2.1 Sample Test Case Details

#	Test Case	Expected Result	Status
1	TC_SIGNUP_API_001: valid user registers via API	201 Created; user in DB	Passed
2	TC_SIGNUP_API_004: reject too-short password	400 Bad Request; validation error	Failed (D001)
3	TC_PERF_002: 500 concurrent users on /items	95% responses <1 s, no crashes	Passed
4	TC_MODEL_PRICE_001: price model R^2 0.97	Measured R^2 =0.97	Passed
5	TC_MODEL_IMG_001: recognition accuracy 94%	Measured Top-1 =94.41%	Passed
6	TC_LOCUST_001: ramp-up to 100 users over 2 min	p95 latency <200 ms; no errors	Passed
7	TC_LOCUST_002: steady 50 users for 30 min	stable CPU/memory; error rate <1%	Passed
8	TC_LOCUST_003: burst to 200 users in 1 min	graceful response; error rate <2%	Passed
9	TC_LOCUST_004: soak test 1 h at 20 users/s	no memory leaks; <1% errors	Passed

6.3 Detailed Test Case Specification

For the full, detailed test case definitions, steps, data and results, see our Google Drive folder:

<https://drive.google.com/drive/folders/1GVM46Euhcb8OaVpzukCzKom41NS1aFmY?usp=sharing>

6.4 Model Performance Metrics

- **Price Prediction (Dubizzle, XGBoost):**

$$R^2 = 0.97, \quad \text{RMSE} = 238,857 \text{ EGP}, \quad \text{MAE} = 122,389 \text{ EGP}.$$

- **Price Prediction (CarDekho, Random Forest):**

$$R^2 = 0.9733, \quad \text{MSE} = 1.57 \times 10^{11}.$$

- **Vehicle Recognition (Stanford Cars, DenseNet201):**

Top-1 Accuracy = 94.41%, Precision = 0.9485, Recall = 0.9441, F1-score = 0.9463.

6.5 Locust Performance Testing Results

- **Average requests/sec:** 180
- **95th percentile latency:** 185 ms
- **99th percentile latency:** 240 ms
- **Error rate:** 0.5%

6.6 Critical Evaluation

6.6.1 Strengths

- *High predictive and classification accuracy* exceeding 94%.
- *Robust front- and back-end validation*, guaranteeing the integrity of the data.
- *Performance resilience* when there is a large concurrent load (up to 200 users).

6.6.2 Weaknesses

- *Edge-case validation gaps* (e.g. obscure password formats).
- *Damage detection* Sometimes it fails under intense glare.
- *Scalability beyond 500 users* needs more stress testing.

6.6.3 Methodology Reflections

- Although it added overhead, the dual toolchain (Katalon + Postman + Locust) allowed for clear separation.
- Rapid development was made possible by FastAPI and Python; performance could be further improved by a compiled language.

6.7 Next Steps for Confidence

- **Extended load tests:** outnumber 1,000 concurrent Locust users.
- **CI/CD integration:** For frequently regression, automate each of the 124 test cases.
- **Adversarial image tests:** low resolution, glare, and artificial occlusions.
- **Resource monitoring:** incorporate automatic memory/CPU threshold alerts.

Chapter 7

Conclusions and Future Work

7.1 Conclusions

In order to help everyone in Egypt, especially those who are not experts, buy and sell used cars with greater confidence, we developed a comprehensive web application for this project. Our main accomplishments are:

- **User Account Management:**
 - Passwords can be recovered, profiles can be edited safely, and users can sign up, log in, and log out.
- **Car Listings:**
 - Authenticated users can create, edit, remove, and browse car listings with photos and additional details.
 - Only the "My Listings" page allows each user to view and control their own ads.
- **Search, Filter and View Details:**
 - Users can find listings by make, model, year, or price with ease thanks to a quick and user-friendly search and filter interface.
 - Full details, the seller's contact information (if allowed), and an image gallery are displayed when you click on a listing.
- **Machine Learning Features:**
 - *Price Prediction:* Fair market prices are predicted by an XGBoost model with $R^2 = 0.97$.

- *Car Recognition*: A classifier based on DenseNet201 can identify the year, make, and model from any photo with an accuracy of about 94% accuracy.
- *Damage Detection*: Users can see necessary repairs by using a YOLOv8-seg network that has been trained on CaDD to highlight dents and scratches.
- **Messaging:**
 - Buyers and sellers have access to real-time chat, conversation history review, and message marking.
- **Admin Dashboard:**
 - Administrators have access to data scraping scheduling, market trend visualization, user and listing management, and platform statistics.
- **Technology Stack:**
 - The system is quick, modular, and simple to expand thanks to its React + Material-UI frontend, FastAPI backend, MongoDB storage, and separate ML services.

All of these components work together to demonstrate how an end-to-end AI-powered solution can manage the marketplace, auto pricing, and user recognition in a straightforward, equitable, and simple manner for all users.

7.2 Future Work

Many ideas were still on the table at the end of the project. The following are some areas that could use improvement:

- **Robust Preprocessing for Damage Detection:**
 - Reduce errors in low-light or harsh sunlight by implementing automatic brightness and contrast adjustment.
 - Include edge enhancement and adaptive noise reduction to improve YOLOv8-seg's ability to detect minor scratches.
- **Fine-Grained Damage Classification:**

- For more precise repair estimates, expand the model to include labels for different types of damage, such as "paint scuff" and "bumper crack."

- **Broader Dataset Collection:**

- Collect more real-world damage photos in varied lighting and angles to make the damage model more robust outside of CaDD.

Future research will incorporate more recent data into the dataset and take into account alternative deep learning architectures. In order to increase the accuracy of vehicle recognition, it will also look into other image processing methods like multi-scale fusion and advanced feature enhancement. The ultimate goal is to create reliable, easy-to-use tools for pricing and recognition that can continuously adjust to changing market conditions.

Appendix A

Collected Materials

Bibliography

- [1] M. Hankar, M. Birjali, and A. Beni-Hssane, "Used Car Price Prediction using Machine Learning: A Case Study," in *2022 11th International Symposium on Signal, Image, Video and Communications (ISIVC)*, El Jadida, Morocco, 2022, pp. 1–4, doi:10.1109/ISIVC56254.2022.9995772.
- [2] A. Amirkhani and A. H. Barshooi, "DeepCar 5.0: Vehicle Make and Model Recognition Under Challenging Conditions," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 1, pp. 541–553, Jan. 2023, doi:10.1109/TITS.2022.3212921.
- [3] X. Ma and A. Boukerche, "An AI-based Visual Attention Model for Vehicle Make and Model Recognition," in *2020 IEEE Symposium on Computers and Communications (ISCC)*, Rennes, France, 2020, pp. 1–6, doi:10.1109/ISCC50000.2020.9219660.
- [4] S. Naseer, S. M. A. Shah, S. Aziz, M. U. Khan, and K. Iqtidar, "Vehicle Make and Model Recognition using Deep Transfer Learning and Support Vector Machines," in *2020 IEEE 23rd International Multitopic Conference (INMIC)*, Bahawalpur, Pakistan, 2020, pp. 1–6, doi:10.1109/INMIC50486.2020.9318063.
- [5] S. Kumar, D. Kaur, and A. Parvez, "Prediction of Prices Car Price Prediction with Machine Learning," in *2022 International Conference on Cyber Resilience (ICCR)*, Dubai, United Arab Emirates, 2022, pp. 1–4, doi:10.1109/ICCR56254.2022.9995772.
- [6] R. Swarnkar, R. Sawant, H. R., and S. P., "Multiple Linear Regression Algorithm-based Car Price Prediction," in *2023 Third International Conference on Artificial Intelligence and Smart Energy (ICAIS)*, Coimbatore, India, 2023, pp. 675–681, doi:10.1109/ICAIS56108.2023.10073882.

- [7] A. Alhakamy, A. Alhowaity, A. A. Alatawi, and H. Alsaadi, "Are Used Cars More Sustainable? Price Prediction Based on Linear Regression," *Sustainability*, vol. 15, no. 2, p. 911, 2023, doi:10.3390/su15020911.
- [8] S. Maity, A. Bhattacharyya, P. K. Singh, M. Kumar, and R. Sarkar, "Last Decade in Vehicle Detection and Classification: A Comprehensive Survey," *Arch. Comput. Meth. Eng.*, 2022.
- [9] M. A. Manzoor, Y. Morgan, and A. Bais, "Real-time Vehicle Make and Model Recognition System," *Mach Learn Knowl Extr*, vol. 1, no. 2, pp. 611–629, 2019.
- [10] Q. Mingjun, L. Guangli, L. Xuejian, M. Xiaolong, and Z. Li, "Structural Description Model for Vehicle Feature Recognition," in *2020 International Conference on Virtual Reality and Intelligent Systems (ICVRIS)*, IEEE, pp. 477–481, 2020.
- [11] D. T. Munroe and M. G. Madden, "Multi-class and Single-class Classification Approaches to Vehicle Model Recognition from Images," in *Proceedings of AICS*, pp. 1–11, 2005.
- [12] J. Sochor, A. Herout, and J. Havel, "BoxCars: 3D Boxes as CNN Input for Improved Fine-Grained Vehicle Recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 3006–3015, 2016.
- [13] Y. Ren and S. Lan, "Vehicle Make and Model Recognition Based on Convolutional Neural Networks," in *2016 7th IEEE International Conference on Software Engineering and Service Science (ICSESS)*, pp. 692–695, 2016.
- [14] A. Dehghan, S. Z. Masood, G. Shu, and E. Ortiz, "View Independent Vehicle Make, Model and Color Recognition Using Convolutional Neural Network," arXiv:1702.01721, 2017.
- [15] F. Tafazzoli, H. Frigui, and K. Nishiyama, "A Large and Diverse Dataset for Improved Vehicle Make and Model Recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition Workshops*, pp. 1–8, 2017.
- [16] W. Liu and Y. Wang, "Monza: Image Classification of Vehicle Make and Model Using Convolutional Neural Networks and Transfer Learning," Stanford University, 2017.

-
- [17] M. H. Zwemer, G. M. Y. E. Brouwers, R. G. J. Wijnhoven, and P. H. N. de With, "Semiautomatic Training of a Vehicle Make and Model Recognition System," in *International Conference on Image Analysis and Processing*, pp. 321–332, 2017.
 - [18] X. Dai, B. Southall, N. Trinh, and B. Matei, "Efficient Fine-Grained Classification and Part Localization Using One Compact Network," in *Proceedings of the IEEE International Conference on Computer Vision Workshops*, pp. 996–1004, 2017.
 - [19] L. Lu and H. Huang, "A Hierarchical Scheme for Vehicle Make and Model Recognition from Frontal Images of Vehicles," *IEEE Trans. Intell. Transp. Syst.*, vol. 20, no. 5, pp. 1774–1786, 2018.
 - [20] Y. Tian, W. Zhang, Q. Zhang, G. Lu, and X. Wu, "Selective Multi-Convolutional Region Feature Extraction Based Iterative Discrimination CNN for Fine-Grained Vehicle Model Recognition," in *2018 24th International Conference on Pattern Recognition (ICPR)*, pp. 3279–3284, 2018.